22. FlashMLA

Chapter Abstract. MLA reduces decode-time cache traffic, but a serving kernel still has to map each logical position to physical cache bytes and preserve the attention result. At its public operator boundary, FlashMLA has four distinct attention paths. Cache-side decode has dense MLA and sparse FP8 MLA paths. Prompt-side prefill has sparse MLA and dense MHA paths. Split decode adds scheduler-metadata and combine kernels around its attention producer, while dense MHA prefill also exposes a backward kernel. One paged-cache request passes through cache addressing, FP8 representation, split reduction, seesaw scheduling, and Hopper overlap. The central test is unchanged throughout: a faster path is interpretable only when it returns the declared output and log-sum-exp within tolerance.

22.1. Why MLA Needs Specialized Kernels

Start from one logical obligation: scaled dot-product attention over the valid cache positions must return the declared result. FlashMLA's physical call must reconstruct that result from page addresses, positional rules, dtype and scale metadata, and any split state. The semantic reference is

$$\mathrm{Attn}(q, K, V) = \mathrm{softmax}\big(\alpha q K^\top\big)\, V.$$

The call also returns the log-sum-exp normalization record used for verification [27].

Absorbed MLA creates an unusual reuse pattern: many query heads read one latent-plus-RoPE cache row. Compression reduces bytes, while head-wise score and value work can raise arithmetic intensity enough to make compute and bandwidth simultaneous constraints.

Definition 22.1.1 (BF16 MLA Decode Arithmetic Intensity) *For h_q query heads, s_q query tokens, s_k cached tokens, key-like width d_k, and value width d_v, a dense BF16 absorbed-decode model uses*

$$F \simeq 2h_q\, s_q\, s_k(d_k + d_v), \qquad M \simeq 2s_k\, d_k,$$

so

$$I_{\mathrm{MLA}} \simeq h_q\, s_q \frac{d_k + d_v}{d_k}.$$

This model applies to the recorded dense BF16 path. Other shapes and cache formats require their own FLOP and traffic accounting.

At $h_q = 128, s_q = 1, d_k = 576, d_v = 512$, the model gives about 241.8 FLOPs/byte. The recorded FlashMLA discussion rounds $d_k + d_v \approx 2d_k$ to about 256 FLOPs/byte; the actual classification still depends on the measured GPU ridge and effective clocks [18].

Symbol or field	Meaning	Toy value
B, S_q, S_{kv}	Batch, query, and cached-token counts	$1, 1, 4$
H_Q, H_{KV}	Query and KV heads	$4, 1$
d_{qk}, d_v	Score and value widths	$6, 4$
P	Cache-page size	2
`block_table`	Logical-to-physical page map	$[7, 3]$
`cache_seqlens`	Valid cached positions	$[4]$
`topk`, k	Sparse selected positions	$k = 2$
out, lse	Attention output and log-sum-exp	Oracle outputs

Table 22.1.: Local notation for the FlashMLA running call.

Definition 22.1.2 (Attention kernel) *An attention kernel is a compiled implementation of a specified map from query and key/value state to attention output, with fixed shape, layout, masking, address metadata, dtype, scales, and reduction semantics.*

The attention-kernel contract in Definition 22.1.2 is instantiated by the chapter's running decode call with $B = 1$, $S_q = 1$, four query heads, one KV head, and four cached positions in two logical pages. A block-table row $[7, 3]$ maps logical pages 0 and 1 to physical blocks 7 and 3. Dense decode visits every valid logical position; sparse decode names a selected set. MLA compression changes each position's representation, whereas sparsity changes which positions are scored.

Trace connection. The source-derived architecture lane of the Volume II request trace, `REQUEST-TRACE/ARCHITECTURE`, supplies the MLA shape used for this projection. Its observed-runtime lane, `REQUEST-TRACE/OBSERVED`, retains 108 FlashMLA calls across V2-Lite attention layers 0–26 and one BF16 cache signature with shape $[47746, 64, 1, 576]$, stride $[36864, 576, 576, 1]$, and page size 64. That is selected-backend and cache-layout evidence, not block-table values, latency, numerical equivalence, or the V3 shape used by the toy projection.

Table 22.1 fixes the toy call used below. The following sections preserve this operator and address contract while changing layout, sparse selection, tiling, split reduction, and measurement.

22.2. Cache Addressing and Sparse Indices

An attention equation indexes token positions. A serving kernel indexes memory. The purpose of a block table is to keep those two coordinate systems aligned when a request's logical sequence is stored in physical cache pages. This is the same kind of problem that paged-cache serving systems solve at runtime: logical token order must remain stable even when physical storage is allocated in page-sized blocks [50]. FlashMLA's dense decode interface receives this mapping as a block table plus cache sequence lengths. Its sparse decode interface can receive already encoded physical entries as sparse cache indices [27].

FlashMLA block-table addressing. Definition 20.7.1 fixes the logical-to-physical allocation contract. FlashMLA consumes that contract as a block-table row for

each batch row i. If token position t has logical block $\lfloor t/P \rfloor$ and offset $t \bmod P$, then a block table value b_{phys} maps that token to physical slot

$$b_{\text{phys}}P + (t \bmod P).$$

Definition 22.2.1 (Cache Length) *For a batch row i, let c_i denote the value* `cache_seqlens`$[i]$. *This value is the number of logical cached token positions that are valid before the current decode call. Dense decode may attend only to positions $0, \ldots, c_i - 1$ for that row, subject to any additional mask. In the running example, $c_i = 4$, so all four logical positions in the table below are valid and no fifth position may be read [27].*

Under the cache-length boundary in Definition 22.2.1, the running example has $P = 2$ and the block table row $[7, 3]$. The logical-to-physical arithmetic is small enough to write out fully.

Table 22.2.: Address conversion for the running cache example.

Logical token t	Logical block	Offset	Physical block	Encoded slot
0	0	0	7	$7 \cdot 2 + 0 = 14$
1	0	1	7	$7 \cdot 2 + 1 = 15$
2	1	0	3	$3 \cdot 2 + 0 = 6$
3	1	1	3	$3 \cdot 2 + 1 = 7$

Figure 22.1 gives the same conversion as a coordinate map: logical token order remains separate from physical slot number.

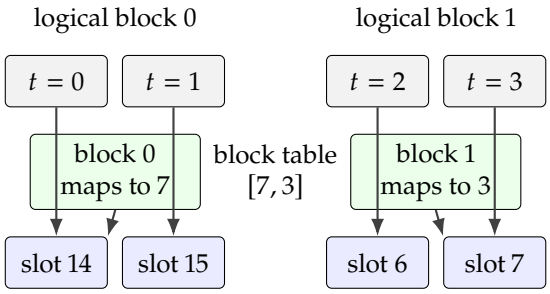

Figure 22.1.: The block table maps logical blocks to physical blocks; the offset inside each block gives the encoded slot used by dense and sparse cache reads.

Coordinate Checkpoint Keep three coordinate systems apart. Logical token positions describe the sequence order that attention semantics use. Block-table entries describe which physical cache block stores each logical block. Encoded physical slots are the page-offset addresses consumed by sparse decode. Dense decode starts from the logical prefix and block table; sparse decode starts after selection has already produced physical slots. A smaller physical slot number is not evidence of an earlier logical token.

Dense decode can use the block table because it walks the valid prefix. With `cache_seqlens` $= [4]$, all four rows in Table 22.2 are valid. Sparse decode instead receives the selected physical slots directly. For top-k set $\{1, 3\}$, the sparse index row is $[15, 7]$. The quantization helper performs this same conversion from absolute logical indices and a block table to the physical index format expected by the sparse kernel [27].

A sparse cache index is already a physical page-offset slot, so the block table is no longer the lookup at that point.

Definition 22.2.2 (Sparse Cache Index) *For batch row i, query row j, and selected-token rank k, `indices_in_kvcache`$[i, j, k]$ is the encoded physical page-offset slot of the selected cache token. Invalid entries are represented by -1. Once this value is supplied, the sparse decode kernel does not require `block_table` for those selected tokens because the physical address has already been encoded.*

Contract 22.2.3 (SM100 sparse-cache address range is contiguously valid)**.** For SM100 sparse attention, every byte from the beginning address of a cache tensor through its final cache byte must be a valid device address. A tensor may be a slice of one larger allocation, because the entire intervening range is still valid. A collection of disjoint page allocations does not satisfy this contiguous-range invariant merely because a logical page table can enumerate those pages [27].

This is stronger than last-dimension contiguity. A tensor view can have the expected shape and byte stride while its backing storage is assembled from separate allocations at a lower systems layer. The SM100 sparse interface documents a range-validity requirement because its address generation may legally visit the represented cache interval. A runtime allocator must prove the storage provenance before launch; discovering the violation through an illegal-memory-access fault is too late.

For example, a slice covering physical blocks 7 through 9 inside one allocation can satisfy the invariant even when the slice does not start at the allocation's base. Three independently allocated blocks with the same tensor contents do not. The logical block table and sparse indices solve token-to-slot identity; they do not manufacture a single valid address interval from disjoint storage.

Example 22.2.1 (Alternate sparse slots with an invalid entry) With page size $P = 2$ and block table row $[7, 3]$, logical position 2 maps to physical block 3 and offset 0, so its encoded slot is 6. Logical position 0 maps to physical block 7 and offset 0, so its encoded slot is 14. A selected-token row for logical positions $\{2, 0, \text{pad}\}$ is therefore $[6, 14, -1]$. The -1 entry is not a cache address; it is a mask request that the reference and kernel must treat as invalid before score reduction.

Two invariants matter in practice. First, address conversion must preserve position identity: logical token 3 cannot become token 2 merely because the physical page identifiers are nonmonotone. In the example, the encoded slot for token 3 is 7, which is numerically smaller than token 1's slot 15, but the logical order is still known by the selected-index construction. Second, invalid entries must be masked before the score reduction. The pinned FlashMLA reference code clamps invalid indices only to make a gather operation legal, then assigns their scores to $-\infty$. A correct kernel and a correct reference implementation must agree on that convention, or they can agree numerically for the wrong reason.

22.3. FP8 KV Cache Layout and Numerical Interpretation

The phrase "FP8 KV cache" does not mean that every cached scalar is stored in the same FP8 format. The FlashMLA source is more precise. In the documented sparse FP8 layout for DeepSeek V3-family dimensions, each cached token stores a quantized non-RoPE component, scale factors, and a BF16 RoPE component. FlashMLA calls the first region NoPE; this chapter follows the book-wide policy of saying non-RoPE except when preserving that source label. The interface comments and quantization helper make this partition visible [27]. The reason to define the layout carefully is that byte counts, bandwidth expectations, and numerical error all depend on which part of the state is quantized.

Definition 22.3.1 (FP8-With-Scale KV Cache) *An FP8-with-scale KV-cache entry is a cache record. It stores a block of non-rotary components as FP8 values together with one or more scale values. The scales encode the dequantization factors for tiles of that block. A position-sensitive RoPE component remains in a higher-precision format. In the FlashMLA layout studied here, the source-backed dimensions are* $d_{\text{nope}} = 512$, $d_{\text{rope}} = 64$, *and* $d_{qk} = 576$.

The byte arithmetic is direct. The documented sparse FP8 layout stores 512 FP8 non-RoPE values, which occupy 512 bytes. It then stores four `float32` scale values, one for each 128-value tile of the non-RoPE part, which occupy $4 \cdot 4 = 16$ bytes. Finally, it stores 64 BF16 RoPE values, which occupy $64 \cdot 2 = 128$ bytes. The total is therefore

$$512 + 16 + 128 = 656$$

bytes per cached token in that layout. As a local comparison, a purely BF16 storage of 576 key-like scalars would occupy $576 \cdot 2 = 1152$ bytes. This comparison is only a byte-count example. It is not a full speedup claim, because wall-clock behavior also depends on memory coalescing, selected token count, scheduling, reductions, and hardware.

Figure 22.2 records the source-backed layout as a byte ledger. The visual grouping matters because the non-RoPE part, scale values, and RoPE part carry different numerical obligations.

The 656-byte example is a mixed-layout ledger: FP8 non-RoPE bytes, scale bytes, and BF16 RoPE bytes have different obligations.

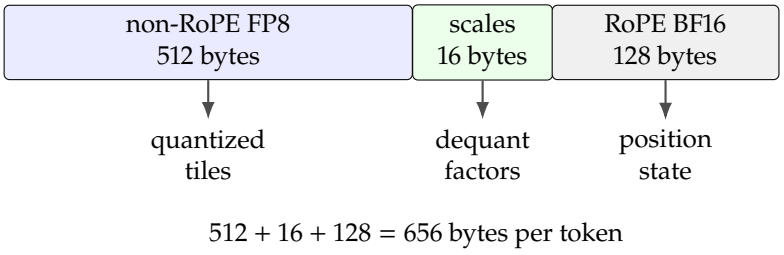

Figure 22.2.: The sparse FP8 cache layout is mixed: quantized non-RoPE values, scale metadata, and BF16 RoPE values contribute separate byte and tolerance terms.

Calling the cache "FP8" hides three separately accounted objects: quantized payload tiles, scale metadata, and BF16 RoPE state. The running example uses the same structure at toy scale. Let the cache record have $d_{\text{nope}} = 4$ and $d_{\text{rope}} = 2$. If the non-RoPE part is split into two tiles of length 2, the cache record has four

quantized non-RoPE entries, two scale entries, and two RoPE entries. The exact byte count depends on the scale dtype chosen for the toy example. The semantic structure is the same: the non-rotary part is quantized with local scales, while the position-sensitive part is not treated as an ordinary FP8 tile.

Definition 22.3.2 (Non-RoPE/RoPE Cache Partition) *The non-RoPE/RoPE cache partition separates the non-rotary coordinates used in the attention score from the rotary position-sensitive coordinates. The non-RoPE part can be tiled and quantized with local scales in the documented FlashMLA sparse FP8 layout. The RoPE part must remain associated with the addressed token's position, and in the recorded layout it remains BF16. In the running example, the split* $d_{\text{nope}} = 4$ *and* $d_{\text{rope}} = 2$ *plays the same semantic role as the source-backed* $512 + 64$ *split [75, 11, 27].*

From Stored Bytes to MMA Operands The byte ledger is only the storage half of the FP8 path. The recorded Hopper sparse-decode kernel does not feed its cached E4M3 bytes directly to an FP8 matrix instruction. It reconstructs BF16 operands first, because the 64 RoPE coordinates already remain BF16 and the released calculation performs both $QK^\top$ and PV with BF16 inputs and FP32 accumulation [17].

Definition 22.3.3 (FlashMLA FP8 Sparse-Decode Execution Path) *For one recorded 656-byte cache row, FlashMLA loads* 512 `float8_e4m3` *non-RoPE values and four FP32 scales. It dequantizes the four 128-value tiles into* 512 *BF16 values. It then concatenates the* 64 *stored BF16 RoPE values for the score operand and executes the score and value matrix products with BF16 inputs and FP32 accumulators. The cache format is FP8 with scales. The MMA input format in this path is BF16.*

On the recorded H800 path, conversion itself is expensive. FlashMLA's sparse-FP8 record traces each E4M3 value through half, FP32, and BF16 conversion before applying its FP32 scale. These CUDA-core operations feed Tensor Cores that can finish the corresponding matrix work quickly. A path is *dequantization-bound* at a shape when producing usable operands takes longer than the matrix instructions that consume them.

Example 22.3.1 (Why sparse FP8 decode can be dequantization-bound) For one CTA handling 64 query heads, that record estimates

$$\frac{64(576 + 512)2}{4096} \approx 34$$

cycles of MMA work per cached token on H800. Its instruction-throughput model estimates at least

$$512\left(\frac{1}{64} + \frac{1}{64} + \frac{1}{16} + \frac{1}{256}\right) \approx 50$$

cycles for the conversions and scale multiplication. The estimates are not a portable timing law, but they identify the local bottleneck: independently dequantizing the full shared row in every CTA would leave the Tensor Cores waiting for CUDA-core conversion work [17].

Definition 22.3.4 (Crossover Dequantization) *Crossover is the recorded Hopper sparse-FP8 schedule in which a cluster of two CTAs handles the same query token, with 64 of its 128 query heads per CTA. Each CTA owns 32 selected-row positions in each 64-position top-k block. It loads and dequantizes the full* 512*-coordinate non-RoPE payload for its assigned rows, copies their* 64 *BF16 RoPE coordinates, keeps those reconstructed rows in its own shared memory, and writes the same rows asynchronously into the other CTA's shared memory through distributed shared memory. After a cluster transaction-barrier handoff, both CTAs possess the full 64-row BF16 operand while each performed only half of the block's payload conversion.*

The crossover dequantization in Definition 22.3.4 describes exchange, not a different attention domain. Both CTAs still attend to the same selected cache tokens and must see the same dequantized tile. The pinned implementation uses 128-bit `__ldg` loads for each assigned row and `st.async` for the remote shared-memory write. The cluster barrier answers when the exchange is complete; it does not replace the numerical requirement that both reconstructed tiles agree [17].

Example 22.3.2 (One full 64-row crossover block) CTA 0 owns query heads 0:63 and selected-row positions 0:31 in the block; CTA 1 owns heads 64:127 and positions 32 : 63. For every assigned row, the owner converts all 512 non-RoPE values and publishes the reconstructed row to its partner. Without sharing, both CTAs would convert all 64 rows, for

$$2 \cdot 64 \cdot 512 = 65{,}536$$

non-RoPE conversions across the pair. Crossover performs

$$32 \cdot 512 + 32 \cdot 512 = 32{,}768,$$

plus the remote BF16 movement needed to give both CTAs the same complete tile. Each CTA can then execute attention for its 64 heads. The pinned source reports 410 TFLOPS for one specified compute-bound sparse shape versus 250 TFLOPS for its earlier kernel without crossover; those values remain shape- and H800-specific reported results, not a general speedup claim [17].

Figure 22.3 makes the ownership exchange and its readiness barrier explicit.

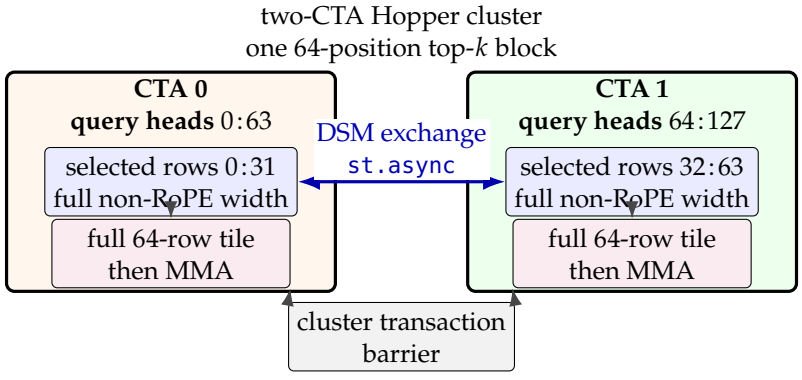

Figure 22.3.: Crossover divides a 64-position selected-row block across two CTAs. Each converts full-width rows and exchanges its 32-row half through distributed shared memory. The cluster barrier makes the full BF16 tile visible before either CTA begins the dependent MMA work.

For the running example, the six score coordinates split into four non-RoPE coordinates and two RoPE coordinates. A cache record can quantize the four

non-RoPE entries in two tiles while keeping the two RoPE entries aligned with token positions $0, \ldots, 3$. With source dimensions, the same partition is 512 non-RoPE values plus 64 RoPE values; changing either side changes the byte count and the comparison tolerance.

The non-RoPE/RoPE distinction comes from attention's positional structure. RoPE is a positional rotation of query and key coordinates [75]. The DeepSeek MLA implementation exposes the configuration fields `qk_nope_head_dim` and `qk_rope_head_dim` in configuration and model code [11]. In FlashMLA's sparse FP8 layout, the RoPE component remains BF16. Thus an FP8 cache is a mixed layout, not uniform quantization of the entire attention state.

Validation Rule 22.3.5 (Layout-aware numerical validation)**.** Any numerical claim about an FP8 MLA cache must state the layout, the scale format, the dimensions of the quantized and unquantized components, and the tolerance used to compare the dequantized result with a reference. A byte count without a layout is incomplete; an accuracy claim without a reference and tolerance is not reproducible.

The pinned FlashMLA source's `quantize_k_cache` and `dequantize_k_cache` helpers are useful because they show the round-trip convention in executable form. They also show that source-specific layouts can differ. Accordingly, the 656-byte calculation is a documented layout for the recorded FlashMLA source, not a universal property of all MLA implementations.

22.4. Dense Versus Sparse MLA

The V3.2 sparse-long-context chapter owns the canonical distinction: MLA changes the representation stored per position, while sparse attention changes the set of positions scored. FlashMLA implements dense and sparse kernel interfaces; it does not make MLA itself a position-selection method [10, 27].

For a query, a dense token set contains every cached position allowed by the decode rule. A sparse token set is a selected subset, represented in FlashMLA sparse decode by physical cache indices. These terms describe the score domain, not the cache entry's dimensions or dtype.

In the running example, dense decode scores positions $\{0, 1, 2, 3\}$. Sparse decode scores top-k positions $\{1, 3\}$, encoded as physical slots $[15, 7]$. The key-like cache row keeps its non-RoPE/RoPE interpretation, and the source-backed sparse layout still keeps FP8 non-RoPE values, scales, and BF16 RoPE values attached. Only the softmax score domain changed.

The counterexamples are decisive. Storing a compressed MLA entry for all four positions and scoring all four is compression without sparsity. Storing ordinary BF16 MHA keys and values but selecting $\{1, 3\}$ is sparsity without MLA. A real sparse FlashMLA call may combine both axes, so its caller must provide valid physical indices and mask invalid entries while the kernel preserves the declared cache layout and precision contract.

DSA owns how the selected positions are constructed. This chapter begins at the kernel boundary: selected indices arrive from the caller, and the output is attention over that declared domain.

22.5. Split-KV Attention and Stable Reduction

A split-KV schedule is correct only when local maxima, normalizers, and weighted values merge over the same logical positions.

The metadata object named `FlashMLASchedMeta` is a useful reminder that a kernel call also carries execution state. The implementation must decide how to partition work across the hardware, how to split long cache reads, how to stage partial reductions, and when previously generated scheduler metadata can be reused. The recorded interface stores the scheduler configuration after the first invocation and checks later calls against that configuration [27].

Each work partition performs ordinary attention over a subset of cache rows: it computes partial scores and weighted values, records the normalization state for that subset, and contributes those records to the final split combine. The CUDA schedule changes where and when those operations run, but not the attention operator they implement.

Definition 22.5.1 (Tile Scheduler Metadata) *Tile scheduler metadata is auxiliary state that records how a FlashMLA decode problem is partitioned for execution. In the Python interface, the metadata object stores a configuration containing batch size, query length, query-head count, page block size, key/value-head count, causal flag, FP8-cache flag, sparse top-k, and optional extra-cache shape information. Reusing the metadata is valid only when the subsequent call agrees with that stored configuration and with the relevant length tensors.*

The metadata also schedules work. Decode batches can contain requests with very different valid lengths, and one long request can expose far more cache-block work than several short requests. The metadata kernel turns dense sequence lengths or sparse top-k lengths into request-and-block jobs, then assigns those jobs so that SMs do not wait behind one disproportionately long row. The exact assignment is implementation state. Every legal split must appear exactly once in the combine domain [18].

Definition 22.5.2 (Programmatic Dependent Launch in FlashMLA) *Programmatic Dependent Launch (PDL) is the recorded dense-decode launch arrangement in which the combine kernel is launched as dependent work for the split-KV kernel. The dependent launch can overlap launch and setup work with the producer's execution. It does not remove the data dependency. Combine may consume a split record only after the producer has made that record available under the CUDA dependency protocol.*

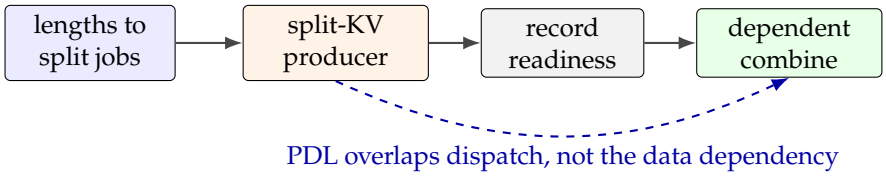

Figure 22.4.: Tile scheduling exposes request-and-block jobs; Programmatic Dependent Launch overlaps producer and dependent-kernel dispatch while the readiness gate still protects split records.

Example 22.5.1 (Balancing one long row and two short rows) Suppose three dense requests expose 64, 8, and 4 cache-block jobs. Assigning one entire request to one SM would leave the two short-request SMs idle while the 64-block row continued. A split scheduler can instead distribute ranges of the long row alongside the short rows, producing several partial (out, lse) records for the long request. PDL can dispatch the dependent combine without a separate host-side launch gap, but that combine must still wait for all records named by the long request's split count. This example shows the scheduling objective without fixing one block-to-SM order.

Figure 22.4 separates launch overlap from the readiness condition that still protects each split record.

The running example has only four query heads and two cache pages, but it still shows the scheduling problem. A dense decode can split work by query head and by cache block. Each piece loads some query coordinates and some cache entries, forms partial scores, and contributes to a reduction that must produce one output vector per query head. Sparse decode changes the cache traversal: instead of scanning the valid prefix, the work is organized around the selected physical slots. In both cases the final result must be independent of how the kernel partitioned the work, up to the documented floating-point tolerance.

Definition 22.5.3 (Split-KV CUDA Schedule) *A split-KV CUDA schedule partitions one logical attention row across multiple cache-block ranges. Each partition produces a partial output accumulator and a partial log-sum-exp value. A later combine kernel rescales those partial outputs by their log-sum-exp weights and writes the final output and lse for the query row.*

The plain action is easier than the implementation vocabulary suggests. Give each split a range of cache rows. For every range, keep enough state to describe its score normalization and its weighted-value result. Then merge the split records into the one answer that would have been obtained had all cache rows been processed together. Nothing in that reasoning requires CUDA syntax.

Softmax must divide every exponentiated score by the same normalizer

$$Z = \sum_j e^{p_j}.$$

Log-sum-exp is simply the logarithm of that normalizer. If $\ell = \log Z$, then each softmax weight can be written as $e^{p_j - \ell}$. Keeping this scalar in the log domain also gives split work a compact normalization record that can be merged without first materializing one potentially enormous exponential sum. As a mental model,

log-sum-exp is a smooth maximum: for n finite scores,

$$\max_j p_j \le \ell \le \max_j p_j + \log n.$$

Definition 22.5.4 (Log-Sum-Exp Return) *The log-sum-exp return value* lse*, written mathematically as* ℓ*, is the value* $\log \sum_j \exp(p_j)$ *for the attention score vector* p *over the selected score domain of one query and head. With* $m = \max_j p_j$*, the equivalent maximum-shifted form is*

$$\ell = m + \log \sum_j e^{p_j - m}.$$

Every shifted exponent is at most 1*, so this form avoids the overflow that can occur when a large positive score is exponentiated directly. In FlashMLA decode,* lse *is returned with shape* (B, H_Q, S_q)*. In the running dense example, each of the four query heads has one log-sum-exp value; in sparse decode the same return shape is computed over only the selected slots* $[15, 7]$*. A correctness test that ignores* lse *checks only part of the interface contract.*

Example 22.5.2 (Large logits without large exponentials) For scores $p = [1000, 999]$, directly forming $e^{1000} + e^{999}$ overflows common FP16, BF16, FP32, and FP64 formats. Maximum shifting instead gives

$$\ell = 1000 + \log(1 + e^{-1}) \approx 1000.313.$$

The weights are $e^{1000-\ell} \approx 0.731$ and $e^{999-\ell} \approx 0.269$. If the two scores came from separate singleton splits, their partial log-sum-exp values would be 1000 and 999; merging those two scalars with the same maximum-shift rule recovers 1000.313.

Suppose one head's scores are split into subsets a and b. Each split may compute

$$\ell_a = \log \sum_i e^{a_i}, \qquad \ell_b = \log \sum_j e^{b_j},$$

The complete attention domain then has the following mathematical log-sum-exp merge across both subsets:

$$\ell = \log \left(e^{\ell_a} + e^{\ell_b} \right).$$

An implementation evaluates the same expression stably with

$$m = \max(\ell_a, \ell_b), \qquad \ell = m + \log \left(e^{\ell_a - m} + e^{\ell_b - m} \right).$$

If o_a and o_b are the separately normalized value accumulators, the matching output merge is

$$o = e^{\ell_a - \ell} o_a + e^{\ell_b - \ell} o_b.$$

Thus the combine step cannot merely add two softmax outputs. It must rescale them using the same global log-sum-exp that it returns to the caller.

Algorithm 4: Stable split-KV attention and completion-gated combine for one row.

Input: Query row q; disjoint valid cache ranges $R_1, \ldots, R_S$; keys K; values V; score scale α
Output: Attention output o and log-sum-exp ℓ

foreach *split* s **do**
  if R_s *is empty* **then**
    publish $(o_s, \ell_s) \leftarrow (0, -\infty)$;
  else
    initialize $m_s \leftarrow -\infty$, $z_s \leftarrow 0$, and $O_s \leftarrow 0$;
    foreach *score/value block* (K_b, V_b) *in* R_s **do**
      set $p \leftarrow \alpha q K_b^\top$ and $m' \leftarrow \max(m_s, \max p)$;
      set $O_s \leftarrow e^{m_s - m'} O_s + e^{p - m'} V_b$ and $z_s \leftarrow e^{m_s - m'} z_s + \sum_j e^{p_j - m'}$;
      set $m_s \leftarrow m'$;
    publish $(o_s, \ell_s) \leftarrow (O_s / z_s,\ m_s + \log z_s)$;
wait until every required split record is published;
if *every* $\ell_s = -\infty$ **then**
  return *reject the empty attention domain*;
set $m \leftarrow \max_s \ell_s$ and $Z \leftarrow \sum_s e^{\ell_s - m}$;
set $\ell \leftarrow m + \log Z$ and $o \leftarrow \sum_s e^{\ell_s - \ell} o_s$;
return (o, ℓ);

Algorithm 4 first assigns each valid cache position to exactly one split and builds a maximum-shifted local normalization record. Then it waits for every required record, so an incomplete split cannot be mistaken for an empty one. Finally, it forms one global log-sum-exp and uses that same value to rescale every partial output. This shared normalizer is why the result is independent of the scheduling partition, within the declared floating-point tolerance, and why adding normalized partial outputs is unsafe.

Implementation lens: FlashMLA's split normalizer

The SM90 combine kernel performs the same maximum-shifted merge in base-2 arithmetic. The excerpt below omits unroll directives, the warp reductions, and the attention-sink branch; the two comments marked "book elision" replace those omitted source ranges.

```
float max_lse = -INFINITY;
for (
    int i = 0; i < NUM_LSE_PER_THREAD; ++i)
    max_lse = max(
        max_lse, local_lse[i]);
// ... book elision: warp maximum reduction ...
max_lse = max_lse == -INFINITY ? 0.0f : max_lse;

float sum_lse = 0;
for (
    int i = 0; i < NUM_LSE_PER_THREAD; ++i)
    sum_lse = sum_lse + exp2f(
        local_lse[i] - max_lse);
// ... book elision: warp sum reduction ...

float global_lse = (
    sum_lse == 0.f || sum_lse == -INFINITY)
    ? INFINITY : log2f(
```

```
        sum_lse) + max_lse;
// ... book elision: natural-log return and attention sink ...
for (
    int i = 0; i < NUM_LSE_PER_THREAD; ++i) {
    const int split_idx = i*32 + lane_idx;
    smem_buf[warp_idx][split_idx] =
        exp2f(
            local_lse[i] - global_lse);
}
```

The first loop finds a common anchor. The second forms the shifted normalizer, and the final loop publishes one rescaling coefficient per split. The later accumulation loop multiplies each normalized partial output by that coefficient. Because `exp2f` and `log2f` use base 2 internally, the kernel converts the returned *lse* to the interface's natural-log convention. Changing the logarithm base changes the representation, not the softmax weights.

This abridged excerpt comes from FlashMLA revision `9241ae3ef9ba`, under the repository's MIT License [27]. The public companion records the exact upstream source coordinate in `chapter_flashmla/README.md`. The excerpt shows the normalizer mechanics, not split publication readiness, attention-sink handling, output conversion, or a performance result.

The compact checkpoint below executes the chapter's 1000-versus-999 two-split example in natural-log arithmetic. It is a constructed semantic reference, not a CUDA-kernel transcription.

Executable checkpoint: Stable two-split log-sum-exp combine

```
from math import exp, log

partial_lse = [1000.0, 999.0]
partial_output = [(1.0, 0.0), (0.0, 1.0)]
maximum = max(partial_lse)
normalizer = sum(exp(lse - maximum) for lse in partial_lse)
global_lse = maximum + log(normalizer)
weights = [exp(lse - global_lse) for lse in partial_lse]
output = tuple(
    sum(
        weight * partial[column]
        for weight, partial in zip(weights, partial_output,
    strict=True)
    )
    for column in range(2)
)

assert round(global_lse, 6) == 1000.313262
rounded_output = tuple(round(value, 6) for value in output)
assert rounded_output == (0.731059, 0.268941)

if __name__ == "__main__":
    print(f"global LSE={global_lse:.6f}; combined
    output={rounded_output}")
```

Run: `python3 chapter_flashmla/split_kv_lse_checkpoint.py`

The full CPU-only reference checks large logits, partition invariance, empty splits, mismatched vector widths, and the all-empty rejection rule. It lives at `chapter_flashmla/`.

22.6. Seesaw Tile Scheduling and Architecture Realization

An ordinary ping-pong schedule keeps two output accumulators. One worker group can then perform matrix work while another performs softmax work on CUDA cores. FlashMLA's absorbed value width makes that duplication prohibitively expensive. One 64×512 FP32 output matrix contains 32,768 32-bit values. The recorded Hopper SM has 65,536 32-bit registers, so two full matrices would consume the entire register file before queries, scores, normalization state, addresses, or compiler temporaries are accounted for [18].

The name *seesaw* describes which warpgroup is doing which kind of work, not where the output lives. Once warpgroup 0 has formed the score fragment P_0, it can use CUDA cores for the maximum and softmax while warpgroup 1 forms P_1 with Tensor Core matrix work. The roles then tip the other way: warpgroup 0 begins the $P_0 V_{0,L}$ value product while warpgroup 1 performs the maximum and softmax work for P_1. Output ownership never tips. Warpgroup 0 keeps O_L, warpgroup 1 keeps O_R, and the probability fragments cross only so that both cache blocks contribute to both fixed halves. This alternation of CUDA-core and Tensor Core work is the performance mechanism; the shared online-softmax state is the correctness constraint [18].

Definition 22.6.1 (Seesaw Schedule) *A seesaw schedule is FlashMLA's two-warpgroup dense-decode schedule that retains one logical output matrix, partitions its value coordinates into $O_L, O_R \in \mathbb{R}^{64\times 256}$, and assigns one half to each warpgroup. Across two cache blocks, the groups reverse the score/softmax and value-product roles while keeping output ownership fixed. Both halves use the same online-softmax maximum and normalizer, so their concatenation is mathematically one attention output, not two independently normalized outputs.*

The seesaw schedule in Definition 22.6.1 is legal only because the online-softmax invariant makes its rearrangement precise. If the current state is maximum m, exponential sum z, and unnormalized accumulator $O = [O_L \; O_R]$, then a new score block p uses

$$m' = \max(m, \max p), \qquad \rho = e^{m-m'}, \qquad P = e^{p-m'},$$

$$z' = \rho z + \sum_j P_j, \qquad O' = \rho O + PV.$$

After the final score block, the returned log-sum-exp and normalized output are $\ell = m + \log z$ and $o = O/z$. The online state is therefore the maximum-shifted representation of the same normalizer defined above, rather than a separate scheduling quantity. Seesaw may compute the left and right columns of PV in different warpgroups, but the same ρ, m', and z' govern both. A barrier handoff

is therefore a mathematical obligation as well as a scheduling event: using a stale maximum for either half would concatenate two incompatible softmax states.

Example 22.6.1 (One output matrix divided between two warpgroups) For cache blocks 0 and 1, write $V_i = [V_{i,L}\ V_{i,R}]$. Warpgroup 0 owns O_L, while warpgroup 1 owns O_R. The four required value products are

$$P_0V_{0,L}, \quad P_1V_{1,L} \longrightarrow O_L, \qquad P_0V_{0,R}, \quad P_1V_{1,R} \longrightarrow O_R.$$

Read the schedule in three moments. First, warpgroup 0 normalizes P_0 while warpgroup 1 forms P_1. Second, warpgroup 0 begins $P_0V_{0,L}$, and warpgroup 1 advances the shared maximum and normalizer for P_1. Third, the older probability fragment and previously accumulated output are rescaled to that newer maximum before the fragments cross: warpgroup 0 receives P_1 for $P_1V_{1,L}$, while warpgroup 1 receives the rescaled P_0 for $P_0V_{0,R}$. Each half occupies $64 \cdot 256 = 16{,}384$ FP32 accumulator values, while the concatenated result remains the same 64×512 matrix required by ordinary online attention.

Figure 22.5 shows the role reversal first and the fixed ownership and cross-group handoffs second. It is a dependency sketch, not a cycle-accurate instruction trace.

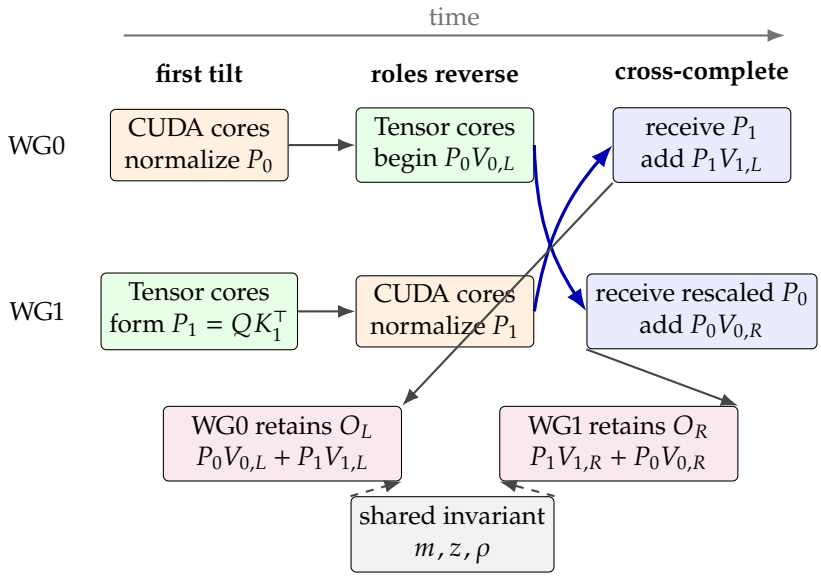

Figure 22.5.: Conceptual seesaw timeline for one pair of cache blocks. Time moves left to right: CUDA-core normalization and Tensor Core matrix work tip between the two warpgroups, then probability fragments cross. Output ownership remains fixed, and both 64×256 halves follow one shared online-softmax state.

Seesaw reverses worker roles while output-half ownership and the shared online-softmax state remain fixed.

The adjective *swizzled* below describes storage, not an attention permutation. A *memory-layout swizzle* rearranges addresses inside the shared-memory staging tile so lane accesses avoid pathological bank conflicts; the logical attention rows and key/value coordinates do not move. That is distinct from *coalescing*, which describes neighboring GPU lanes combining nearby global-memory requests into efficient transactions. A load can need both a coalesced global access pattern and a swizzled shared-memory destination.

The source-backed dense decode path maps that sequence onto a pipeline for one 64×64 cache tile. In the Systems Foundations model, the cache page is the global tensor, its descriptor and tile coordinates select the requested rectangle, the shared stage is the destination, and the transaction barrier is the byte-readiness record. First, the *Tensor Memory Accelerator* (TMA) copies a rectangular key tile from

global memory into a swizzled shared-memory buffer. The copy is asynchronous: issuing it starts movement, and the consumer waits on its transaction barrier before using the buffer. Next, the *warpgroup matrix multiply-accumulate* (WGMMA) instruction family multiplies the query fragment by the staged keys to form a $QK^\top$ score fragment. The kernel applies the score transformations and updates the row's online-softmax maximum and sum.

The staged cache page also supplies the value coordinates; WGMMA multiplies the probability fragment by that value region to accumulate PV. While that work consumes one shared-memory buffer, TMA can fill another with a later cache tile. Barrier waits and the pipeline handoffs prevent an early read or reuse of a buffer whose previous work is still in flight [27].

Figure 22.6 makes that dense SM90 path spatial. The upper plate follows one page from HBM into the active stage, through score and value work, and onward as a split record that must be recombined. The lower strip shows only that filling page $t + 1$ can overlap matrix work on page t; horizontal length is not elapsed time.

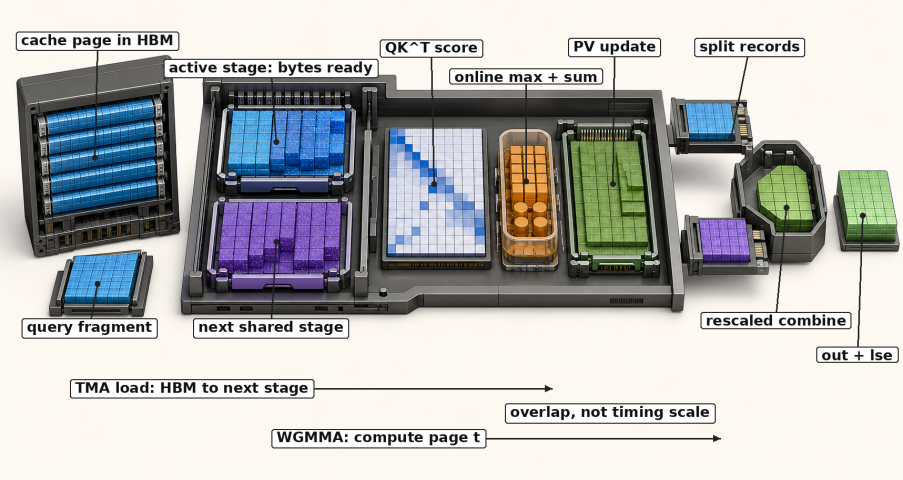

Figure 22.6.: Conceptual mechanism plate for one SM90 dense-decode cache tile. A TMA load moves a page from HBM toward the next shared stage; the transaction wait makes that stage readable. WGMMA uses the active stage for $QK^\top$ and PV; online maximum-and-sum state contributes to each split record, and the combine stage rescales partial outputs by their log-sum-exp values before out and lse are complete. The geometry and overlap strip are neither a literal hardware floorplan nor an instruction-count or timing scale.

The two arrows do not collapse three completion facts. A TMA transaction-barrier wait makes the promised bytes readable; the WGMMA fence, commit, and wait sequence orders and completes the tracked matrix groups; and a named CTA barrier hands shared online-softmax state between worker groups. The logical scheduler view below keeps the metadata gate and split-combine topology exact.

The `cp.async` helpers in FlashMLA's SM80 utility header solve the same staging problem at a smaller granularity: GPU threads issue asynchronous global-to-shared copies and wait before consuming the copied data. They are not an extra step in the SM90 dense tile above. On Hopper, TMA performs the rectangular tile transfer and WGMMA performs the matrix products. The sparse FP8 path preserves the same attention sequence but adds selected-index validity, dequantization state, and per-block softmax updates before accumulating $S@V$.

Figure 22.7 abstracts away the spatial mechanism and keeps the scheduling argument exact. The metadata gate defines which tiles or selected slots are legal; the data path then preserves the log-sum-exp normalization through score fragments, value accumulators, and split recombination.

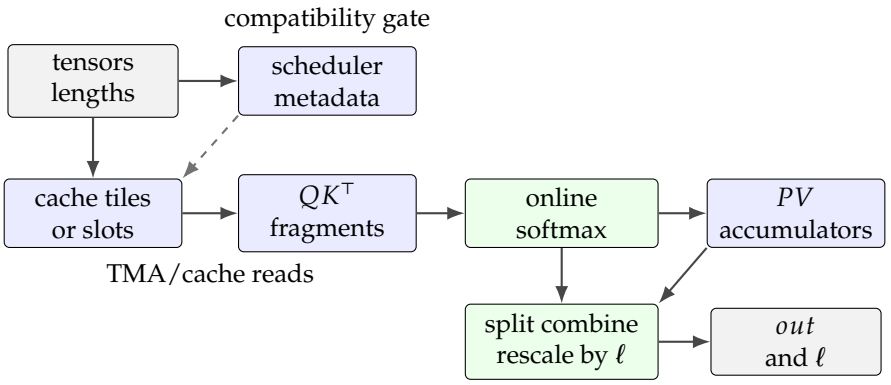

Figure 22.7.: FlashMLA scheduling connects the launch metadata to tiled cache movement, score products, online softmax state, value accumulators, and split recombination.

Table 22.3 connects the metadata object, cache layout, and returned lse to their correctness roles. The table covers representative CUDA surfaces; FlashMLA's CUDA sources also contain separate SM90 dense decode, SM90 sparse-FP8 decode, SM90 sparse prefill, SM100 decode, and SM100 dense or sparse prefill paths.

Architecture-specific realization of a FlashMLA tile

The instruction family changes with the target GPU, but each implementation must move cache bytes, establish when they are safe to consume, compute score and value products, and preserve the online-softmax state. FlashMLA's SM80 utilities expose `cp.async` copies for global-to-shared staging. The SM90 dense path groups rectangular copies under TMA barriers and uses WGMMA for $QK^\top$ and PV. The SM100 utilities add TMA gather, tensor-memory, and `tcgen05` helpers for that architecture; Section 22.11 follows their two-CTA sparse-prefill schedule and lazy-anchor normalization state. Sparse-FP8 decode also uses non-coherent, cache-hinted global loads for its selected cache records. These are different realizations of the same FlashMLA obligations; none changes which positions belong to the attention row or how split normalization is combined [27, 66].

Two jobs, three completion facts. The pipeline overlaps two jobs. TMA fills the next shared-memory buffer while WGMMA consumes the current buffer and updates fragments held in the participating lanes' registers. A transaction-barrier wait answers whether the promised bytes have arrived. The WGMMA fence, commit, and wait sequence answers whether collective matrix work is correctly ordered and sufficiently complete. A named CTA barrier answers the separate question of whether the two warpgroups may exchange shared online-softmax state.

Chapter 19 defines *mbarrier* and gives a six-step double-buffer trace in its reusable transaction-pipeline section. That material introduces byte readiness; the preceding WGMMA and completion sections there separate fence, issue, commit, and wait. None changes the attention domain, the returned output, or lse. The three completion conditions stay distinct in the pinned SM90 source-to-hardware case. Chapter 35 follows the nine-slice cache pipeline beside the other measured NVIDIA cases. Reused scheduler metadata remains valid only for the matching request and launch contract.

CUDA object	Source-visible role	Invariant
Metadata kernel	A 32-thread kernel computes request ranges, split counts, and `DecodingSchedMeta` rows from dense sequence lengths or sparse top-k lengths.	Split metadata is part of the launch contract, not a runtime afterthought.
TMA tile copies	Key/value tiles are copied into shared memory with barriers; dense decode uses page-block tiles, while sparse FP8 decode uses selected top-k blocks and validity masks.	Cache address metadata must agree with the bytes the kernel stages.
WGMMA products	Warpgroup matrix operations compute score fragments $QK^\top$ and value products SV.	The attention equation is tiled, but the output must match the same reference domain.
Online softmax state	Register values track per-row maxima and sums while tiles are visited.	Partial tiles must merge by log-sum-exp, not by adding unnormalized softmax outputs.
Combine kernel	A separate CUDA kernel gathers split accumulators, merges partial lse values, rescales output fragments, and writes final `out`.	A split launch is incomplete until its combine step has preserved both output and lse.
Seesaw warpgroups	Two warpgroups own the 64×256 left and right output halves and exchange probability or normalization state.	Both halves must use the same online maximum, sum, and rescale history.
Dependent launch	The combine kernel is launched under PDL after the split-KV producer.	Dispatch overlap cannot make an incomplete split record readable.

Table 22.3.: FlashMLA CUDA internals that support the dense and sparse decode contract.

22.7. Repository Surfaces and Kernel Interfaces

The pinned FlashMLA source names one realization of the address, representation, selection, and reduction contracts already derived.

With the operator contract fixed, the pinned FlashMLA source becomes a realization map. FlashMLA exposes dense and sparse decoding, sparse MLA prefill, and dense MHA prefill through a PyTorch extension module [27]. The interface at the source-record snapshot separates the Python call contract from the architecture-specific CUDA implementation. Model code must satisfy the call's tensor, cache-layout, and sparse-index conditions before any fast path is eligible.

Python imports `flash_mla.cuda` and calls exported functions such as `sparse_decode_fwd`, `dense_decode_fwd`, and `sparse_prefill_fwd`. Under the extension-module definition in Systems Foundations, these pybind exports determine how the runtime loads the package and locates its callables; they do not assert a dispatcher namespace.

Kernel hierarchy at the pinned revision. The four lifecycle surfaces do not collapse to one CUDA kernel. Decode assigns most attention arithmetic to a

split-KV producer and then requires scheduler and combine kernels to make that partitioned work complete. Sparse prefill and dense MHA prefill instead use separate forward launch families, and dense MHA also exposes a backward family. Table 22.4 orients those principal and supporting launchers. Here *principal* means the launcher that owns the surface's attention arithmetic in the recorded call graph, not a universal claim about which symbol dominates runtime for every shape.

Table 22.4.: Principal and supporting FlashMLA kernel launchers at the pinned revision. Public lifecycle surfaces name the operator boundary; source launchers identify the implementation family that realizes it.

Surface	Principal source launcher	Role and supporting work
Dense MLA decode	`run_flash_splitkv_mla_kernel`	Tiled $QK^\top$, online softmax, and PV over dense page metadata.
Sparse FP8 MLA decode	`run_flash_splitkv_mla_fp8_sparse_kernel`	The same fused attention sequence over selected slots, with FP8 reconstruction and Hopper crossover.
Shared decode support	`run_get_decoding_sched_meta_kernel` `run_flash_mla_combine_kernel`	The scheduler assigns legal splits; combine merges partial output and LSE. Both are required for split decode.
Sparse MLA prefill	`run_fwd_phase1_kernel`	Gather selected rows and return out, `max_logits`, and LSE; SM100 may select a small-top-k phase-1 variant.
Dense MHA prefill	`run_fmha_fwd` `run_fmha_bwd`	Variable-length dense MHA: forward returns o, LSE; backward maps saved state and d_o to d_q, d_k, d_v.

Five interface layers divide the call, binding, kernel, quantization, and correctness responsibilities (Table 22.5).

Concept-to-code map

The wrapper boundary begins at `https://github.com/deepseek-ai/FlashMLA/blob/9241ae3ef9bac614dd25e45e507e089f888280e0/flash_mla/flash_mla_interface.py`; C++ and sparse-forward surfaces share the pinned revision [27].

- **Decode scheduling and reduction.** The public `FlashMLASchedMeta`, `get_mla_metadata`, and `flash_mla_with_kvcache` route to the dense or sparse split-KV launcher together with `run_get_decoding_sched_meta_kernel` and `run_flash_mla_combine_kernel`.
- **Sparse ownership and prefill call chain.** The caller or indexer owns selection; `sparse_attn_decode_interface` consumes decode slots. For prefill, `flash_mla_sparse_fwd` calls the `sparse_prefill_fwd` export bound to `sparse_attn_prefill_interface`. The dispatcher chooses `run_fwd_phase1_kernel` or an eligible SM100 `run_fwd_for_small_topk_phase1_kernel`. Regular SM90 and SM100

Interface layer	Technical responsibility
Python call interface	Decode and prefill signatures, tensor shapes, dense or sparse branch conditions, scheduler metadata, and return values.
Build and C++ binding	Extension construction and pybind exports for the callable FlashMLA functions.
Dense and sparse decode headers	Architecture, dtype, contiguity, page-size, head-dimension, and sparse-feature eligibility checks.
Kernel utilities and CUDA sources	Tensor Memory Accelerator (TMA) and warpgroup matrix multiply-accumulate (WGMMA) helpers, sparse-FP8 dequantization loads, and SM90/SM100 implementation families.
Quantization helpers	FP8 cache layouts, quantize/dequantize operations, and logical-token to physical-cache index conversion.
Reference and kernel tests	Reference attention computations, numerical tolerances, invalid-index behavior, and validated shape families.

Table 22.5.: FlashMLA interface layers and their technical responsibilities.

paths launch `sparse_attn_fwd_kernel`; the eligible small-top-k path launches `sparse_attn_fwd_for_small_topk_kernel`.

- **Dense MHA prefill.** The variable-length wrappers route forward and backward through `run_fmha_fwd` and `run_fmha_bwd`.
- **Extension loading.** The build and pybind surfaces expose `flash_mla.cuda` directly to Python.
- **Architecture paths.** Scheduler and memory movement map to the SM80, SM90, and SM100 CUDA utilities, including sparse-FP8 decode helpers.
- **SM100 sparse-prefill state.** The regular head-128 phase-1 kernel maps four-row selected loads to `tma_gather4_cta_group_2`, paired score and value products to the two-CTA `tcgen05` helpers, and lazy-anchor normalization to the pinned implementation variables `mi`, `li`, `real_mi`, and the tensor-memory output-rescale subroutine.
- **Dense Hopper scheduling.** The compute-intensity analysis, seesaw ownership split, fine-grained TMA pipeline, and dependent combine map to the dense Hopper split-KV implementation and its public design record.
- **Sparse FP8 crossover.** The mixed cache record, dequantization-cost model, two-CTA exchange, and cluster handoff map to the sparse-FP8 dequantization and split-KV components.
- **Correctness.** Reference attention and kernel outputs are compared by FlashMLA's pinned reference helpers and kernel tests.

The running example instantiates this interface with a query tensor, cache tensor, block table, cache sequence lengths, sparse indices, scheduler object, and output tensors. The quantization helper fixes the FP8 cache byte layout; the reference helper fixes the sparse-attention calculation. At the model boundary, the latent state follows the MLA definitions from DeepSeek-V2 and DeepSeek-V3 [10, 13]. RoPE position handling follows RoFormer and the DeepSeek-V3 non-RoPE/RoPE split [75, 11], while page-table addressing uses the paged-cache abstraction from serving systems [50].

22.8. Dense and Sparse Decode Contracts

The preceding mechanism sections fixed the score domain, cache representation, and reduction rule. The pinned call surface now shows where those objects cross an implementation boundary. Dense decode walks the valid cached prefix; sparse decode names selected physical cache slots directly. The central FlashMLA decode surface has two entry points for that work: the metadata call `get_mla_metadata` and the decode call `flash_mla_with_kvcache`. In the recorded source, `get_mla_metadata` returns a `FlashMLASchedMeta` object and a legacy second return value. The actual scheduling metadata is generated on the first compatible decode call and then stored inside the metadata object.

The decode function consumes a query tensor and a cache tensor. Its address input is either dense page metadata or sparse physical indices. It also receives the value dimension, scheduler metadata, optional `sm_scale`, and a small set of mode-specific arguments. It returns the attention output and the softmax log-sum-exp tensor [27].

Definition 22.8.1 (MLA Decode Surface) *The MLA decode surface is the call contract that maps*

$$(q,\ k_{\text{cache}},\ \text{address metadata},\ d_v,\ \text{scheduler}) \longmapsto (o,\ \ell),$$

where q has shape (B, S_q, H_Q, d_{qk}), o has shape (B, S_q, H_Q, d_v), and ℓ is the log-sum-exp tensor with shape (B, H_Q, S_q). The address metadata is either dense page metadata or sparse physical cache indices.

Definition 22.8.2 (Dense Decode And Sparse Decode) *Dense decode is the decode mode in which the score domain for each query row is the full valid cached prefix recovered from `block_table` and `cache_seqlens`. Sparse decode is the decode mode in which the score domain is a caller-selected set of encoded physical cache slots supplied in `indices_in_kvcache`. In the running example, dense decode scores logical positions $\{0, 1, 2, 3\}$, while sparse decode with top-k set $\{1, 3\}$ scores only the physical slots $[15, 7]$. The source-backed interface contract is that dense decode must receive dense page metadata, whereas the recorded sparse decode path requires sparse indices, `causal=False`, and `is_fp8_kvcache=True` [27].*

Definitions 22.8.1 and 22.8.2 fix the common outputs and the two address domains. Dense decode is the mode closest to ordinary cached attention. The kernel receives a block table and one cache sequence length per batch row. It attends only to valid cached positions. In the running example, `cache_seqlens` = [4] says that logical positions $0, 1, 2, 3$ are valid. The dense mode can therefore recover all four positions from the block table row $[7, 3]$. The Python interface enforces the corresponding condition: when sparse indices are absent, dense decode must provide the block table and cache sequence lengths. Sparse-only arguments must be absent [27].

Sparse decode changes the address metadata. Instead of asking the kernel to walk the whole valid cache sequence, the caller supplies `indices_in_kvcache`, a tensor whose last dimension names selected physical cache entries. In the recorded FlashMLA interface, the sparse branch requires `causal=False` and

`is_fp8_kvcache=True`. This is not a universal theorem about sparse attention; it is a source-backed property of the recorded sparse-decode interface. The caller contract is stricter than the mathematical softmax equation. The selected token set, physical cache layout, FP8 flag, and invalid-index convention must all agree. Only then can the kernel be treated as a correct implementation of the intended attention map.

Dense metadata walks a valid prefix; sparse metadata names selected physical slots directly.

Decode-mode invariant checkpoint. Dense and sparse decode keep the same query and output shapes, but they change the metadata that names the score domain. Dense decode fixes a valid prefix through block-table and sequence-length metadata. Sparse decode fixes selected physical slots through `indices_in_kvcache` and its invalid-entry convention. Neither mode name is a latency result until shape, dtype, cache layout, hardware, command, and correctness gates are recorded.

BF16 and FP16 both use two payload bytes, but they spend their bit budgets differently: BF16 keeps a wider exponent range, while FP16 provides finer local spacing where both formats cover the values. The names are therefore not interchangeable even when an API accepts either one.

Pinned eligibility realization. The C++ API headers narrow those mathematical contracts further. In the recorded dense decode API, the implementation requires SM90a. It accepts BF16 or FP16 queries and caches with dtype agreement. It also requires int32 sequence lengths and block tables, contiguous last dimensions, page block size 64, $d_{qk} \in \{576, 512\}$, $d_v = 512$, and divisibility of query heads by key/value heads. In the sparse decode API, the implementation requires MQA-style $H_{\mathrm{KV}} = 1$, BF16 queries, FP8/int8/uint8 cache storage, and int32 sparse indices. The remaining checks are $H_Q \in \{64, 128\}$, $d_{qk} \in \{576, 512\}$, $d_v = 512$, and SM90a or SM100f execution [27]. These are source-specific implementation constraints. The mathematical definitions above are broader, but a caller using this FlashMLA checkout must satisfy the narrower source preconditions.

Two sparse cache scopes, independent truncation, and an attention sink A sparse decode request can name two cache scopes at the pinned revision. The primary scope is the pair (`k_cache`, `indices_in_kvcache`). An optional second scope is the pair (`extra_k_cache`, `extra_indices_in_kvcache`). The second pair is all-or-nothing: an extra cache without extra indices is not a valid call, nor are extra indices or an extra length valid without the extra cache. The two scopes may use different page-block sizes and different top-k capacities, and those dimensions become part of the cached scheduler configuration [27].

Let the valid primary slots for batch row b be

$$I_b = \{i_{b,0}, \ldots, i_{b,L_b-1}\},$$

and the valid extra slots be

$$I'_b = \{i'_{b,0}, \ldots, i'_{b,L'_b-1}\}.$$

Here L_b comes from `topk_length` when supplied and otherwise equals the primary index width; L'_b is defined independently by `extra_topk_length`. Entries after either length are excluded even when their index values would otherwise be valid. After invalid entries and the two length masks are applied, the reference concatenates the gathered primary and extra rows and computes one softmax over $I_b \parallel I'_b$. Thus the scopes have independent truncation but *joint normalization*. Computing two independently normalized outputs and adding them is a different operator.

Without a sink, softmax assigns all normalization mass to the selected cache rows. FlashMLA's optional attention-sink logit adds a null alternative: algebraically, it acts like one extra softmax candidate with logit a_h and a zero value vector. The candidate can absorb probability mass without adding to the value-weighted numerator, so it attenuates that head's attention output. Here *attention sink* means this explicit FlashMLA argument, not another mechanism or an observed token-level attention pattern.

The optional `attn_sink` has one FP32 value a_h per query head. If the ordinary sparse scores for a query and head have log-sum-exp ℓ_h, the returned attention output is multiplied by

$$\gamma_h = \frac{e^{\ell_h}}{e^{\ell_h} + e^{a_h}} = \frac{1}{1 + e^{a_h - \ell_h}}.$$

The returned `softmax_lse` remains ℓ_h; the sink changes the output normalization mass without changing that diagnostic return. In particular, $a_h = -\infty$ has no effect and $a_h = +\infty$ produces a zero output for that head. A correctness oracle must therefore compare both the unmodified LSE and the sink-scaled output [27].

Figure 22.8 summarizes the two call contracts. Dense metadata reconstructs a valid logical prefix; sparse metadata arrives after selection and names physical slots directly. The optional sparse scope changes the selected set, while the sink changes only the returned attention output.

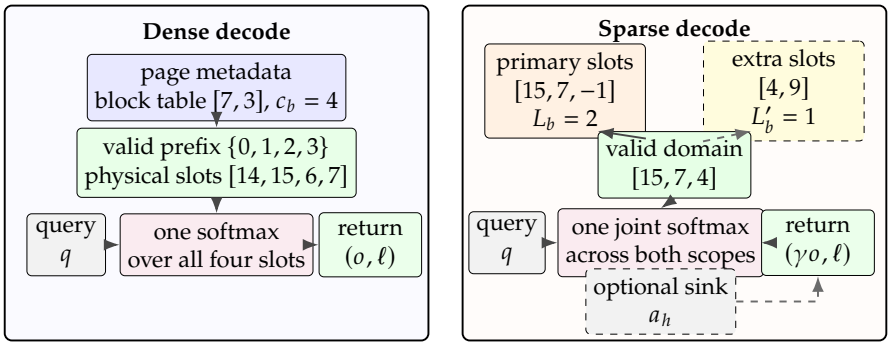

Figure 22.8.: Dense and sparse decode keep the query and return shapes but change the metadata that defines the score domain. Dense decode resolves every valid logical-prefix position through page metadata. Sparse decode consumes selected physical slots, masks invalid or truncated entries, and jointly normalizes the valid primary and optional extra scopes. The optional sink scales o while the returned ℓ remains unchanged.

Example 22.8.1 (Two scopes with different valid lengths) Suppose the primary index row is $[15, 7, 14]$ with $L_b = 2$, while the extra row is $[4, 9]$ with $L'_b = 1$. The score domain is the three physical slots $[15, 7, 4]$. Slots 14 and 9 are excluded by their respective length records; they are not restored merely because the other scope has remaining capacity. If a sink is present, its extra denominator term competes with the single softmax over these three slots, not with two per-scope softmaxes.

22.9. A Worked Sparse FP8 Decode Trace

The compact contract in Section 22.8 isolates the masking rule. The following source-scale example carries that rule through address conversion, the mixed cache layout, crossover dequantization, split reduction, and sink scaling. It deliberately stays on the sparse FP8 decode path. The seesaw schedule in Section 22.6 belongs to the dense Hopper path and is not part of this execution trace.

Example 22.9.1 (One FP8 sparse-decode row from selected tokens to returned tensors) **Setup.** Let $B = 3$, $S_q = 1$, $H_Q = 128$, $H_{\mathrm{KV}} = 1$, $d_{qk} = 512 + 64 = 576$, and $d_v = 512$. These values stay inside the recorded sparse-decode shape family. Follow batch row $b = 1$. Its primary cache uses page size $P = 64$ and block-table row $[7, 3, 12]$. Before the FlashMLA call, the runtime selects logical positions

$$[1, 65, 130, 188, 67].$$

Address conversion. Encode the logical positions as physical slots:

$$[7\cdot 64+1,\ 3\cdot 64+1,\ 12\cdot 64+2,\ 12\cdot 64+60,\ 3\cdot 64+3] = [449, 193, 770, 828, 195].$$

The sparse interface receives these encoded slots rather than the block table. Suppose an already encoded extra-cache scope contributes $[160, 33]$. Let the primary and extra index tensors have last-dimension capacities $k = 128$ and $k' = 1024$, with the displayed slots at their left edges. With primary and extra valid lengths $L_b = 5$ and $L'_b = 2$, the one score domain is

$$I_b \,\|\, I'_b = [449, 193, 770, 828, 195] \,\|\, [160, 33].$$

Any capacity entries after those lengths remain excluded.
Cache reconstruction and crossover. Assume both scopes use the documented mixed sparse-cache record. Each selected row contains 512 FP8 non-RoPE bytes, four FP32 scales occupying 16 bytes, and 64 BF16 RoPE values occupying 128 bytes. The seven selected rows therefore name

$$7(512 + 16 + 128) = 7 \cdot 656 = 4592$$

cache bytes before index and control metadata. The Hopper schedule nevertheless processes selected positions in 64-position blocks. Within each block, CTA 0 owns query heads 0:63 and selected-row positions 0:31; CTA 1 owns heads 64:127 and positions 32:63. Each CTA reconstructs the full 512-coordinate non-RoPE vector and copies the 64 BF16 RoPE coordinates for every row it

owns, then publishes those complete rows into its partner's distributed shared memory. After the cluster transaction barrier, both possess the same 64-row BF16 tile and form BF16 matrix operands with FP32 accumulation. In this seven-row trace the undisplayed capacity positions remain invalid, so the example exposes the ownership rule rather than a balanced 32-versus-32 valid-row workload [27, 17].

Split reduction. To expose the combine invariant without claiming one implementation-specific job assignment, suppose three legal partial records cover the seven selected rows. The recorded scheduler may choose a different split count. For one query head, show two coordinates of each separately normalized partial output:

$$\begin{aligned} s=0: &\quad \ell_0 = 1000, && o_0 = [1, 0], \\ s=1: &\quad \ell_1 = 998, && o_1 = [0, 1], \\ s=2: &\quad \ell_2 = 999.5, && o_2 = [0.25, 0.75]. \end{aligned}$$

With $m = \max_s \ell_s = 1000$, the stable combine computes

$$\ell = m + \log \sum_s e^{\ell_s - m} = 1000 + \log(1 + e^{-2} + e^{-0.5}) \approx 1000.555.$$

The rescaling weights are approximately $[0.5741, 0.0777, 0.3482]$, so

$$o = \sum_s e^{\ell_s - \ell} o_s \approx [0.6611, 0.3389].$$

Adding the three partial outputs without these weights would implement a different operator.

Sink and returned contract. For a head-specific attention-sink value $a_h = 1001$,

$$\gamma_h = \frac{1}{1 + e^{a_h - \ell}} \approx 0.3905, \qquad o_{\text{sink}} = \gamma_h o \approx [0.2582, 0.1323].$$

The returned tensor shapes are

$$o : (3, 1, 128, 512), \qquad \texttt{softmax\_lse} : (3, 128, 1).$$

The sink scales the full 512-coordinate output for this head; the two displayed coordinates are only a diagnostic slice. Its returned LSE remains the pre-sink value $\ell \approx 1000.555$. A reference oracle must therefore reproduce the physical-slot validity mask, dequantization convention, one joint softmax, stable split merge, and output-only sink scaling.

Each partial output has its own log-sum-exp anchor; combine must rescale the partials before adding them.

Figure 22.9 compresses the example into one execution trace. The two-CTA cluster pattern repeats inside the scheduled split jobs; the diagram shows the 64-position schedule block that contains the seven valid rows rather than seven feature-wise row splits.

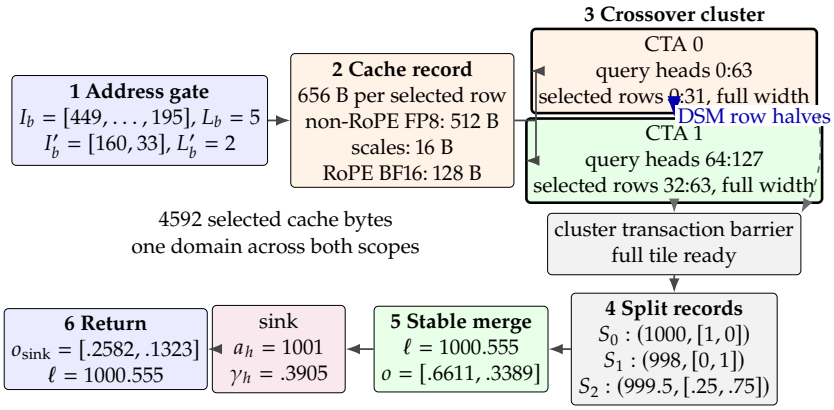

Figure 22.9.: End-to-end sparse FP8 FlashMLA decode for one highlighted batch row. Independent valid lengths define one concatenated physical-slot domain; each 64-position schedule block is cooperatively reconstructed by a two-CTA crossover cluster whose members exchange full-width row halves; split records are rescaled by a stable joint log-sum-exp; and the attention sink scales the output while leaving the returned LSE unchanged.

22.10. Architecture Eligibility and API Mapping

The worked trace establishes one correct sparse path, but it cannot authorize every device, shape, dtype, or lifecycle mode. The dispatch boundary must first state which public surface is eligible; only then can an API or source map be interpreted as the implementation of that surface.

Surface	Architecture and shape family	Additional eligibility boundary
Dense decode	SM90a; $d_{qk} \in \{512, 576\}$, $d_v = 512$	Dense page metadata; BF16/FP16 query and cache agreement; page block 64.
Sparse decode	SM90a or SM100f; $H_Q \in \{64, 128\}$, $d_{qk} \in \{512, 576\}$, $d_v = 512$	MQA cache head, BF16 query, FP8-compatible cache bytes, sparse physical indices, noncausal mode.
Sparse prefill	SM90a or SM100f; $H_Q \in \{64, 128\}$, $d_{qk} \in \{512, 576\}$, $d_v = 512$	BF16 query and packed KV rows; SM100 selects a separate small-top-k implementation when its feature predicate requires it.
Dense MHA prefill	SM100 implementation family; instantiated $(d_{qk}, d_v) = (128, 128)$ and $(192, 128)$ paths	Packed variable-length BF16 rows; backward currently requires equal query and KV head counts.

Table 22.6.: Architecture and shape eligibility of the four FlashMLA lifecycle surfaces at the pinned revision. These are implementation gates, not general limits of attention.

Table 22.6 is the dispatch gate for the four public lifecycle surfaces; later kernel traces must stay inside the architecture, shape, dtype, and mode row they claim to explain.

FlashMLA's scheduler reuse is value-sensitive. The recorded tensor shapes and the values of `cache_seqlens` must remain unchanged. The valid lengths for both sparse scopes must remain unchanged as well. A buffer with the same shape but different valid lengths therefore needs fresh scheduler state.

Concept-to-code map

Dense and sparse decode enter through `https://github.com/deepseek-ai/FlashMLA/blob/9241ae3ef9bac614dd25e45e507e089f888280e0/csrc/api/dense_decode.h` and `https://github.com/deepseek-ai/FlashMLA/blob/9241ae3ef9bac6`

`14dd25e45e507e089f888280e0/csrc/api/sparse_decode.h` at the pinned revision [27].

- **Decode-mode switch.** Dense versus sparse dispatch maps to `flash_mla_with_kvcache`, `indices`, and `indices_in_kvcache` in the recorded FlashMLA source.
- **Dense decode constraints.** Dense call requirements map to `dense_attn_decode_interface` in the FlashMLA C++/CUDA source.
- **Sparse decode constraints.** Sparse dtype, head-count, and cache-index requirements map to `sparse_attn_decode_interface` and `DecodeFeatures` in the FlashMLA C++/CUDA source.

Figure 22.10 places dense and sparse decode in the same call frame. The query tensor and output shapes stay fixed; the metadata selects the score domain.

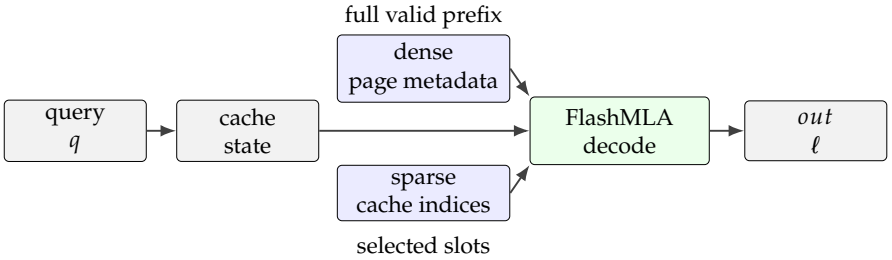

Figure 22.10.: Dense and sparse decode share the query and output shapes, while different metadata selects the score domain.

Figure 22.11 uses the same call-frame logic in a raster base: query and cache inputs are shared, while the metadata choice changes which score domain is read.

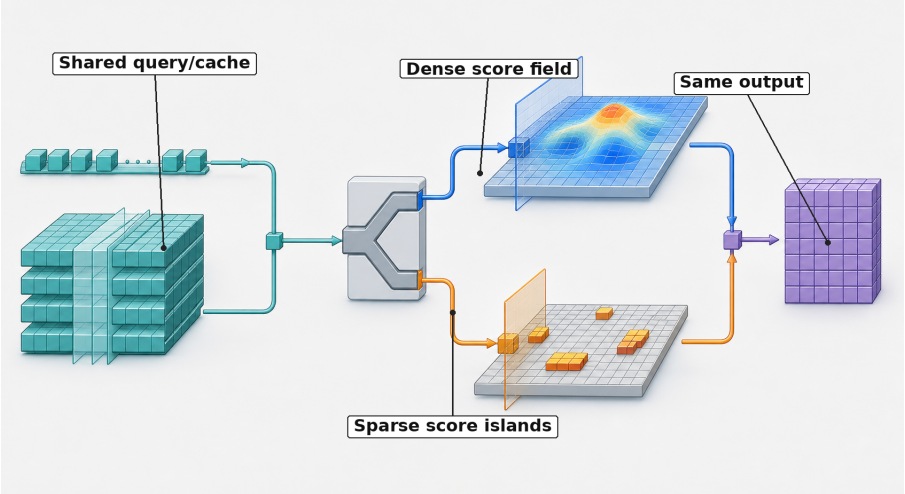

Figure 22.11.: Dense and sparse FlashMLA decode modes share the query and cache inputs, but their metadata selects different score domains before both modes return the same output shape.

Table 22.7 summarizes the two decode modes as API requirements rather than as abstract attention equations.

Table 22.7 is a two-branch call trace: the dense branch sends q, the cache tensor, a `block_table` row such as $[7, 3]$, and a cache length of 4. The sparse branch sends the same q and cache tensor with selected cache positions $[15, 7]$, noncausal sparse decode, and an FP8 KV-cache mode. Both branches cross the same API boundary and return o and ℓ; only the metadata-selected score domain changes.

For the toy decode step, q has shape $(1, 1, 4, 6)$. A dense call should return o with shape $(1, 1, 4, 4)$ and ℓ with shape $(1, 4, 1)$. If the same example is expressed as

Object	Dense decode	Sparse decode
q	(B, S_q, H_Q, d_{qk})	(B, S_q, H_Q, d_{qk})
Cache tensor	$(N_{\text{blk}}, P, H_{\text{KV}}, d_{qk})$	Same source-level cache argument, with sparse FP8 layout requirements in the recorded interface
Address metadata	`block_table` and `cache_seqlens`	`indices_in_kvcache`, plus optional sparse length controls
Mode requirement	Sparse-only arguments absent	`causal=False` and `is_fp8_kvcache=True` in the recorded source
Implementation constraints	SM90a, BF16/FP16, $P = 64$, $d_v = 512$, $d_{qk} \in \{576, 512\}$	SM90a/SM100f, BF16 query, FP8/int8/uint8 cache, $H_{\text{KV}} = 1$, $H_Q \in \{64, 128\}$, $d_v = 512$
Return	$o : (B, S_q, H_Q, d_v)$, $\ell : (B, H_Q, S_q)$	Same shapes

Table 22.7.: API requirements for the two FlashMLA modes used in this chapter.

sparse decode with top-k set $\{1, 3\}$, the returned shapes do not change. What changes is the score domain: dense decode scores four cached positions per query head, while sparse decode scores two. Ignoring the softmax overhead, a simple multiply-add count for dense attention in the toy example is

$$H_Q \left(2Td_{qk} + 2Td_v\right) = 4(2 \cdot 4 \cdot 6 + 2 \cdot 4 \cdot 4) = 320$$

floating-point operations for the score and value products. Sparse top-k with $k = 2$ changes the same denominator to

$$H_Q \left(2kd_{qk} + 2kd_v\right) = 4(2 \cdot 2 \cdot 6 + 2 \cdot 2 \cdot 4) = 160.$$

The two counts are local arithmetic denominators: they show why the selected token set must be recorded before a performance number can be interpreted.

22.11. Prefill Interface Matrix: Sparse MLA and Dense MHA

Prefill and decode stress different shapes. Decode usually has a small query length and a growing cache; prefill processes many query positions while building or reading the prompt-side attention state. FlashMLA exports two surfaces for those roles. Sparse MLA prefill is represented by `flash_mla_sparse_fwd`. Dense MHA prefill is represented by variable-length FlashAttention-style wrappers [27, 5].

The similar names hide two different ways to reduce attention cost. A model mechanism such as DeepSeek Sparse Attention (DSA) changes the logical score domain: its lightning indexer chooses a set S_i of positions for query i. A sparse FlashMLA kernel consumes that choice and executes attention over the selected MLA rows. Standard dense FlashAttention instead changes how a declared domain is executed: it tiles score and value work, carries online softmax state, and avoids materializing the full score and probability matrices in high-bandwidth

memory. It still evaluates every position admitted by the dense call's causal, padding, or window rule [5, 22].

Definition 22.11.1 (FlashAttention Execution Boundary) *Let D_i be the positions admitted for query i by a call's causal and padding rules. In its standard exact form, FlashAttention computes* $\mathrm{Attn}(q_i, K_{D_i}, V_{D_i})$ *with an I/O-aware tiled schedule; up to floating-point effects, the schedule changes data movement rather than the mathematical attention domain. FlashAttention-3 (FA3) specializes that execution idea to Hopper with asynchronous data movement, warp specialization, and low-precision paths. Neither name by itself supplies DSA's learned top-k set. The original FlashAttention work also gives a block-sparse extension, so a backend name alone does not prove that a particular call is dense: the mask or index contract must still identify the domain [5, 73].*

Table 22.8 maps the model-side selector to two kernel-side execution contracts. DSA owns model-side selection; FlashMLA executes dense and sparse modes; FlashAttention follows the domain specified by its mask or indices.

Table 22.8.: DSA selection, FlashMLA sparse execution, and standard dense FlashAttention change different objects. Shared tiling or online-softmax techniques leave their score-domain contracts unchanged.

Object	Who fixes the score domain?	Execution obligation	Primary saving
DSA lightning indexer	The trained model ranks eligible prior positions and emits S_i	Select positions for the main MLA path before final attention	Fewer positions enter main attention, with additional indexer and selection cost
FlashMLA sparse prefill or decode	The caller supplies selected logical or physical indices	Gather source-specific MLA cache rows and compute attention over S_i	Skip unselected main-attention pairs while handling irregular addressing and cache layout
Standard dense FlashAttention or FA3	The call supplies D_i through sequence lengths and masks, without a DSA top-k index set	Tile and fuse attention over every position in D_i; FA3 changes the Hopper schedule and precision options	Reduce score-matrix materialization and HBM traffic; the dense score count remains $\lvert D_i \rvert$

Example 22.11.1 (Four positions: pair-count savings versus IO savings) Suppose a causal query can legally read $D_i = \{0, 1, 2, 3\}$, while the DSA indexer selects $S_i = \{1, 3\}$. A standard dense FlashAttention call evaluates the four legal query–key scores and computes the dense-domain result without storing a four-entry score or probability row in high-bandwidth memory. The sparse FlashMLA call receives S_i, gathers rows 1 and 3, evaluates two main-attention scores, and normalizes only over those selected rows. Its result is exact for the selected domain, not generally equal to the dense-domain result. The two-versus-four main-attention count also does not predict a twofold latency improvement: DSA still pays indexer and selection work, and the two kernels have different addressing, tiling, and launch costs.

Architecture versus kernel execution and prefill versus decode are independent axes. Prefill and decode use different interfaces and state layouts. Table 22.9 makes the four call signatures explicit.

Lifecycle cell	Interface and position record	Required state distinction
Dense prefill	Interface: variable-length dense MHA wrapper. Position record: cumulative sequence lengths over explicit q, k, v tensors.	Dense prompt attention over explicit tensors
Sparse MLA prefill	Interface: `flash_mla_sparse_fwd`. Position record: per-query `indices` into packed kv rows.	Prompt rows selected by the caller
Dense decode	Interface: `flash_mla_with_kvcache` without sparse indices. Position record: `block_table` and `cache_seqlens` recover the valid prefix.	Valid cached prefix reconstructed from page metadata
Sparse decode	Interface: `flash_mla_with_kvcache` with `indices_in_kvcache`. Position record: selected physical cache slots plus sparse-mode flags.	Selected cached positions, distinct from prefill

Table 22.9.: Dense/sparse and prefill/decode are four lifecycle cells, not one attention interface with different names.

Consequently, a cache-side decode test cannot stand in for prompt-side prefill coverage, even when both paths call attention kernels.

Figure 22.12 shows the same boundary as a two-axis lifecycle grid. The rows distinguish prompt-side prefill from cache-side decode; the columns distinguish dense traversal from caller-selected sparse positions.

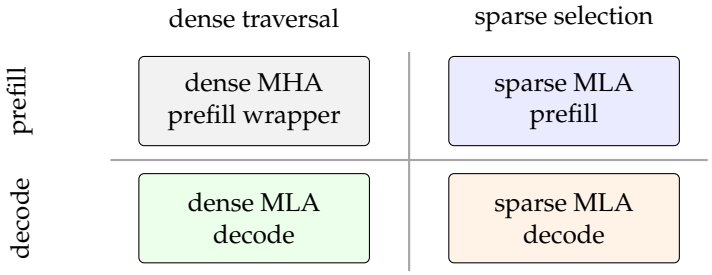

Figure 22.12.: The FlashMLA interfaces live in four lifecycle cells: prefill versus decode changes the state ownership, while dense versus sparse changes how the score domain is named.

Definition 22.11.2 (Sparse MLA Prefill) *Sparse MLA prefill is the call*

$$\texttt{flash\_mla\_sparse\_fwd}(q, kv, \texttt{indices}, s, d_v, \ldots),$$

where q has shape (S_q, H_Q, d_{qk}), kv has shape (S_{kv}, H_{KV}, d_{qk}), `indices` selects a top-k set of key/value rows for each query token, and s is the score scale. The return values are the output, `max_logits`, and the log-sum-exp tensor.

Definition 22.11.3 (Dense MHA Prefill Wrapper) *A dense MHA prefill wrapper is a variable-length dense-attention interface that consumes explicit query, key, and value tensors, cumulative sequence lengths, and maximum sequence lengths rather than an MLA decode cache and page metadata. It represents the ordinary prompt-side dense MHA problem. Within this chapter, the wrapper is a contrast case: it can share the same attention equation, but it does not define `indices_in_kvcache` or the MLA decode cache layout [27].*

Table 22.10 compares the prefill surfaces in Definitions 22.11.2 and 22.11.3 at the call boundary.

Sparse MLA changes which positions are admitted; dense FlashAttention changes how the admitted positions are evaluated.

Table 22.10.: Boundary between FlashMLA sparse MLA prefill and dense MHA prefill surfaces.

Question	Sparse MLA prefill	Dense MHA prefill wrapper
State supplied	q, packed kv, selected-index tensor, score scale, and d_v	Explicit query, key, and value tensors with variable-length sequence metadata
Position rule	Caller supplies the selected key/value rows for each query token	Dense wrapper attends over the represented variable-length sequences
Return check	Output, maximum logits, and log-sum-exp must match the sparse reference	Output must match the dense variable-length attention reference
Failure if confused	A dense wrapper cannot validate sparse top-k selection or invalid sparse entries	A sparse MLA prefill call is not a decode cache walk with larger S_q

The implementation call graphs preserve that boundary. At the pinned revision, sparse prefill dispatches `run_fwd_phase1_kernel`; eligible SM100 small-top-k shapes may instead dispatch `run_fwd_for_small_topk_phase1_kernel`. The public sparse-prefill call returns final out, `max_logits`, and LSE tensors without entering the decode scheduler-plus-combine graph. Dense MHA wrappers route separately through `run_fmha_fwd`; autograd backward combines the saved forward state with the upstream d_o and routes through `run_fmha_bwd`. These are source launcher names, not promises that every specialization emits one identical device symbol.

Figure 22.13 makes the two launch graphs visible.

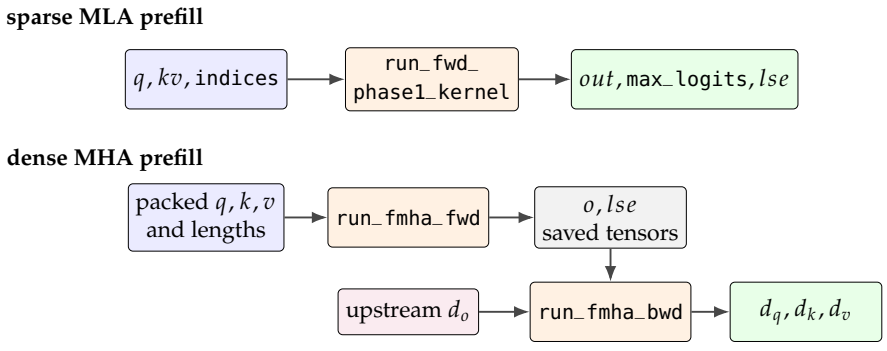

Figure 22.13.: The two prefill launch graphs at the pinned revision. Sparse MLA prefill follows caller-selected rows through a phase-1 forward launcher and returns three forward records. Dense MHA prefill follows packed variable-length state through a forward launcher, then uses saved state and d_o for its separate backward launcher.

The released dense-MHA training interface The dense prefill surface is not forward-only. FlashMLA exports three variable-length wrappers: separate q, k, v, a packed qkv tensor, and a separate q with packed kv. All three reach the same autograd function. The forward pass allocates a 32 MiB byte workspace, writes o and an FP32 LSE tensor, and saves q, k, v, o, ℓ plus both cumulative-length arrays for backward. The backward receives an upstream d_o, reconstructs its

own workspace from batch, aligned maximum query length, head count, and head dimension, and writes d_q, d_k, d_v [27].

Definition 22.11.4 (Dense-prefill gradient semantics) *For the scalar loss $L = \langle o, g \rangle$ with an upstream tensor $g = d_o$, the dense-prefill gradient rule is*

$$(q, k, v, lengths, g) \longmapsto (d_q, d_k, d_v),$$

where the gradients equal those of the same variable-length, scaled, masked attention reference. Forward LSE is saved to reconstruct the softmax derivative, but an upstream derivative for the returned LSE is discarded: the public wrapper does not currently define backward through LSE itself.

The released dense-prefill boundary is narrow. The wrappers assert zero dropout and reject `deterministic=True`; that flag is not a promise of a deterministic backward mode. Backward also raises an error when the number of query heads differs from the number of KV heads, even though forward can cover grouped-query configurations. At the pinned revision the CUDA binding names the forward and backward implementations as SM100 surfaces, and the instantiated BF16 paths use head-dimension pairs $(128, 128)$ and $(192, 128)$. These are released implementation constraints, not claims that the mathematics of dense MHA requires them [27].

Wrapper	Payload layout	Shared training obligation
`flash_attn_varlen_func`	Separate packed-row q, k, v plus query and KV cumulative lengths	Save q, k, v, o, ℓ; compare o, ℓ, d_q, d_k, d_v with the variable-length reference.
`flash_attn_varlen_qkvpacked_func`	One tensor sliced into query, key, and value coordinates	Preserve slice boundaries and return gradients to the corresponding packed regions.
`flash_attn_varlen_kvpacked_func`	Separate q, packed kv, and distinct query/KV lengths	Preserve the packed key/value split and the separate query and KV sequence maps.

Table 22.11.: The three released dense-prefill wrappers share the same forward/backward semantics while presenting different packing layouts.

Table 22.11 distinguishes the three public packing layouts. A small gradient oracle can use two packed sequences with `cu_seqlens_q` $= [0, 2, 3]$ and `cu_seqlens_k` $= [0, 3, 5]$. Construct BF16 q, k, v, choose a fixed FP32 upstream g, and evaluate $L = \sum o \odot g$ once through the wrapper and once through a straightforward PyTorch attention calculation for each sequence. The acceptance record must compare o and ℓ first, then d_q, d_k, d_v under the dtype-aware absolute, relative, and cosine-difference policy in the pinned FlashMLA tests. It must also include negative cases for nonzero dropout, requested deterministic mode, and backward GQA. A forward-only match does not establish the released training surface.

22.12. SM100 Sparse Prefill: Gather and Lazy-Anchor Softmax

The gather-then-attend equation does not by itself explain how an irregular top-k row set becomes a Tensor Core operand. In the pinned regular SM100 head-128 sparse-prefill path, one 128-head query row and one 128-position selected block are assigned to a pair of CTAs. A TMA `gather4` operation names four noncontiguous source rows per issue, so the memory engine can stage caller-selected rows without first packing them through a separate global-memory kernel. The pair then uses `tcgen05` matrix operations with `cta_group::2`; the barriers distinguish gathered-byte readiness from matrix completion [27, 66].

Definition 22.12.1 (Two-CTA Gathered Sparse-Attention Schedule) *For the pinned regular SM100 head-128 sparse-prefill path, a two-CTA gathered schedule assigns 64 query heads to each CTA and processes selected positions in 128-position blocks. For the key operand, CTA 0 gathers selected-row positions* 0:63 *and CTA 1 gathers positions* 64:127*, with each TMA issue naming four rows. For the 512-coordinate value operand, CTA 0 stages coordinates* 0:255 *and CTA 1 stages coordinates* 256:511*. Paired QK^{T} and SV matrix operations consume those complementary pieces as one logical attention tile.*

Figure 22.14 keeps the two different partitions visible. Keys are divided by selected-row position because the score product spans the 128-position block. Values are divided by output coordinate because the value product returns a 512-coordinate vector. Neither partition changes the selected score domain.

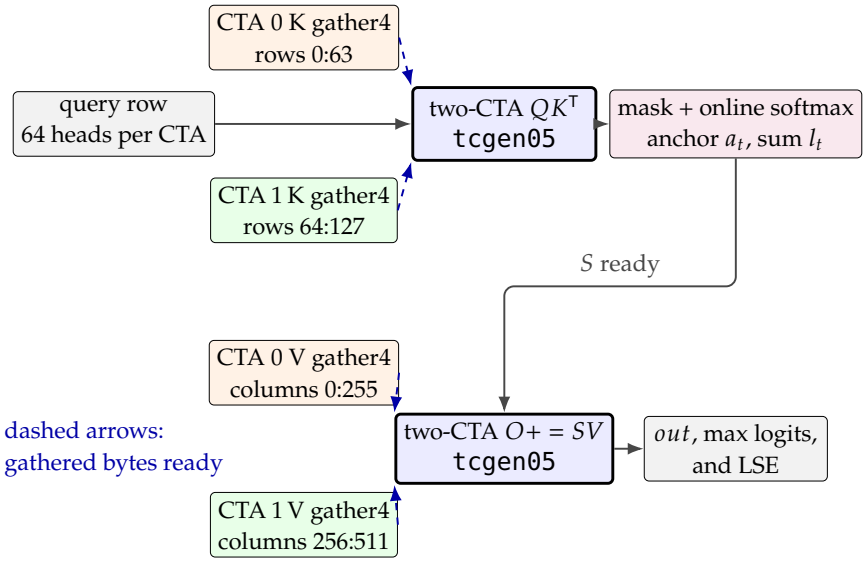

Figure 22.14.: Conceptual source trace for the pinned regular SM100 head-128 sparse-prefill tile. Four-row TMA gathers stage complementary K-row and V-coordinate partitions; transaction barriers gate two-CTA score and value matrix operations. The diagram is a dependency and ownership view, not a cycle-accurate timeline.

The pinned SM100 implementation assigns separate worker groups to K movement, V movement, matrix issue, and scale-and-exponential work. Two shared-memory buffer sets let the memory workers prepare a later block while matrix and softmax workers consume the current one. The runtime sequence is therefore: gather a legal selected block, wait for its promised bytes, form QK^{T}, mask and normalize the scores, wait for the corresponding value coordinates, accumulate

SV, and only then release the buffers for reuse. A gather completion is a gate, not attention work; a matrix completion is a different gate.

Lazy-anchor rescaling in the SM100 online softmax. Conventional online softmax moves its numerical anchor whenever a new block maximum increases. On this SM100 path, moving the anchor can require loading, scaling, and storing the 512-coordinate output accumulator in tensor memory. The pinned source reduces those passes by allowing the exponent anchor to lag the true running maximum within a bounded range.

r is the true maximum and a is the exponent anchor; rescaling both l and O preserves the normalized result.

Definition 22.12.2 (Lazy-Anchor Online-Softmax State) *For one row, let r be the true maximum of every valid score seen so far in base-2 exponent units, let a be the anchor used for exponentiation, and let*

$$l = \sum_j 2^{x_j - a}, \qquad O = \sum_j 2^{x_j - a} v_j.$$

The normalized output is O/l, and the natural-log LSE is $a \ln 2 + \ln l$, regardless of whether $a = r$. The pinned SM100 policy updates r for every block. It enters a collective rescale path when any row in the controlled warp cohort has a new block maximum more than six base-2 units above its anchor. On that path, each row sets $a' = \max(a, u)$ for its own block maximum u. Multiplying both l and O by $2^{a-a'}$ preserves that row's normalized output and LSE [27].

We call this source-visible policy *lazy-anchor rescaling*; the name describes the invariant and is not a public FlashMLA API term. The threshold keeps a newly processed score at most six base-2 units above the retained anchor when the collective path is skipped, so its unnormalized exponential is at most $2^6 = 64$. The SM100 policy lifts the update predicate to the cooperating warp: if any controlled row needs a rescale, every lane follows the same tensor-memory control path. A peer row may then move its anchor by six or fewer units; a row with no maximum increase uses scale factor one.

Algorithm 5: Cohort-voted lazy-anchor online softmax for SM100 sparse attention, omitting the optional attention-sink term.

Input: Valid score/value blocks $(x_{t,b}, V_b)$ for rows t in one controlled cohort, with scores in base-2 exponent units
Output: Per-row normalized output o_t, true maximum r_t, and natural-log LSE ℓ_t

initialize every $a_t \leftarrow -10^{30}$, $r_t \leftarrow -\infty$, $l_t \leftarrow 0$, and $O_t \leftarrow 0$;
foreach *valid score/value block* b **do**
  mask invalid positions; set $u_t \leftarrow \max_j x_{t,b,j}$ and $r_t \leftarrow \max(r_t, u_t)$ for every row t;
  set $g \leftarrow \bigvee_t [u_t - a_t > 6]$ using the cohort vote;
  foreach *row* t **do**
    if g **then**
      set $a'_t \leftarrow \max(a_t, u_t)$ and $\beta_t \leftarrow 2^{a_t - a'_t}$;
      set $l_t \leftarrow \beta_t l_t$, $O_t \leftarrow \beta_t O_t$, and $a_t \leftarrow a'_t$;
    set $S_{t,b} \leftarrow 2^{x_{t,b} - a_t}$;
    set $l_t \leftarrow l_t + \sum_j S_{t,b,j}$ and $O_t \leftarrow O_t + S_{t,b} V_b$;
set $\ell_t \leftarrow a_t \ln 2 + \ln l_t$ and $o_t \leftarrow O_t / l_t$ for every row t;
return $(o_t, r_t \ln 2, \ell_t)_t$;

Algorithm 5 separates four phases. It first masks invalid positions before finding each row's block maximum. It then updates every true maximum unconditionally, preserving the returned `max_logits`. The cohort vote decides whether all participating lanes enter the rescale path, while the scale itself remains row-specific. Finally, each row's block probabilities and value product are accumulated on its selected anchor. This is not an approximate softmax rule: only floating-point rounding distinguishes it from eagerly moving every row's anchor after every increase.

Example 22.12.1 (Deferring two eager rescalings into one) Suppose a row has anchor $a = 10$, and no peer row triggers the cohort vote. A later block has maximum 13, only three base-2 units above the anchor, so the source keeps $a = 10$; its largest unnormalized exponential is $2^{13-10} = 8$. A subsequent block has maximum 17, which crosses the threshold. The old l and O are then multiplied once by

$$2^{10-17} = \frac{1}{128}.$$

An eager policy would have multiplied them first by $2^{10-13} = 1/8$ and then by $2^{13-17} = 1/16$. The cumulative factor is the same $1/128$, while the lazy policy avoids the intermediate tensor-memory rescale.

The mathematical reference for sparse prefill is a gather-then-attend calculation. For a query row i, let I_i be the selected index set. After masking invalid entries, the reference gathers the selected key and value rows before attention:

$$K_i = kv[I_i, :, : d_{qk}], \qquad V_i = kv[I_i, :, : d_v],$$

then computes scaled scores $P_i = s q_i K_i^\top$, masks invalid selected positions to $-\infty$, computes a stable log-sum-exp $\ell_i = \log \sum_j \exp(P_{i,j})$, and returns

$$o_i = \sum_j \exp(P_{i,j} - \ell_i) V_{i,j}.$$

This is the same structure used by the pinned FlashMLA reference tests, with the source-specific details of shapes, invalid indices, optional attention sink, and top-k lengths attached at the call boundary [27]. The sink follows the null-candidate and output-scaling semantics introduced for decode in Section 22.8; on this sparse-prefill surface, it likewise leaves the returned lse and `max_logits` unchanged.

A two-coordinate numerical example makes the reduction concrete. Suppose one query is $q = [1, 0]$, the selected keys are $k_1 = [1, 0]$ and $k_3 = [0, 1]$, the selected values are $v_1 = [2, 0]$ and $v_3 = [0, 4]$, and the scale is 1. The scores are $[1, 0]$. The log-sum-exp is $\log(e^1 + e^0) \approx 1.313$, so the softmax weights are approximately $[0.731, 0.269]$. The reference output is therefore

$$0.731[2, 0] + 0.269[0, 4] \approx [1.462, 1.076].$$

A sparse prefill correctness test should reproduce this logic at source-sized shapes and compare the kernel's output, maximum logits, and log-sum-exp values within a dtype-aware tolerance.

Dense MHA prefill is a different surface. The variable-length wrappers consume explicit query, key, and value tensors together with cumulative sequence lengths and maximum sequence lengths. That interface is useful because the standard dense prefill problem is well described by packed variable-length attention. It is not the same as the MLA decode surface, where the source-level argument named `k_cache` carries the cached MLA state and where dense or sparse address metadata determines which cache entries are read. The caller cannot infer the correct kernel from the word "attention" alone. It must know whether the request is prefill or decode, dense or sparse, MHA or MLA-mode, and what cache representation is present.

For a dense prefill wrapper, two prompts of lengths 3 and 2 can be packed into five explicit q, k, and v rows. The cumulative sequence record is `cu_seqlens` $= [0, 3, 5]$, and the maximum sequence length is `max_seqlen` $= 3$. Output rows 0–2 map back to the first prompt, and rows 3–4 map back to the second prompt. No `block_table` or `indices_in_kvcache` is present, because the call is not walking a decode cache or selecting sparse MLA cache slots.

22.13. Correctness Testing and Numerical Tolerance

Correctness tests must keep logical positions, latent cache state, and numeric tolerance together.

Kernel correctness and kernel performance are separate measurements. A timing row needs a correctness gate tied to the same mode, shape, dtype, and layout. A correctness test states the mathematical reference, constructs mode-specific inputs, runs the kernel, and compares each promised return value under the declared tolerances. The pinned FlashMLA tests follow this pattern. Dense decode is compared with a PyTorch reference over block-table cache rows. Sparse decode is compared with a gathered sparse reference, while sparse prefill checks output, maximum logits, and log-sum-exp. Quantization helpers make the FP8 cache layout explicit [27].

The general reference-implementation and tolerance contracts are defined in Chapter 32. Table 22.12 specializes them to the logical cache state, selected positions, masks, and auxiliary returns required by FlashMLA.

Each surface follows the canonical lifecycle: validate metadata and allocation provenance before launch, reconstruct the logical state, compute every promised reference field, run the kernel, and compare values plus structural invariants. The FlashMLA-specific rule is that a passing output tensor cannot hide a wrong mask, address map, LSE, maximum-logit return, gradient, or cache round trip.

The new rows require structural tests as well as value comparisons. A two-scope sparse-decode fixture should choose different primary and extra lengths, include a valid index after each length, and use values for which separate per-scope softmaxes visibly disagree with one concatenated softmax. Run it once without a sink, once with a finite sink, and once with $+\infty$. The expected LSE is identical between the sink variants while the output changes. Reuse of the same scheduler object after changing either length value should also be rejected by the test harness.

Allocation eligibility belongs before the CUDA call. Construct one accepted fixture as a view into a single larger allocation and represent its byte interval explicitly. Construct the rejected fixture as equal-shaped pages from disjoint

Table 22.12.: Correctness surfaces for FlashMLA reference comparisons.

Surface	Reference must reconstruct	Compared fields and risk caught
Dense decode	Block-table cache rows, cache sequence lengths, head grouping, and masking convention	out and lse; catches page lookup, mask, and shape mistakes
Sparse decode	Encoded physical slots, invalid-entry mask, FP8-cache layout, and selected score domain	out and lse; catches stale indices, unmasked padding, and layout mismatches
Two-scope sparse decode	Primary and extra cache rows, each scope's invalid mask and length, concatenated score domain, and optional sink	out and unmodified lse; catches per-scope normalization, cross-scope masking, and sink/LSE mistakes
Sparse prefill	Gathered key/value rows, top-k lengths, score scale, maximum logits, and log-sum-exp	Output, max logits, and lse; catches reduction and selected-row errors
Dense prefill forward/backward	Packed sequence boundaries, dense attention reference, fixed upstream d_o, and saved o, ℓ	o, ℓ, d_q, d_k, d_v; catches sequence-boundary, saved-state, and gradient errors
FP8 cache round trip	Quantized non-RoPE tiles, scale metadata, dequantization rule, and BF16 RoPE partition	Dequantized cache rows; catches byte-layout and scale interpretation errors
SM100 allocation eligibility	Backing allocation interval for each primary and extra cache, plus tensor start and last-byte addresses	Accept one contiguous allocation or its slice; reject disjoint-page assembly before launch

allocations. The rejection oracle is the runtime adapter's provenance check; the book does not recommend launching the invalid case and waiting for an illegal-memory-access failure.

For sparse decode in the running example, the reference first converts the selected logical positions $\{1, 3\}$ to physical slots $[15, 7]$. It gathers the two selected cache entries. Invalid entries use a harmless gather index, while their scores are masked to $-\infty$. The reference then computes scaled scores for each query head and $\ell = \log \sum_j e^{p_j}$. It forms normalized weights $\exp(p_j - \ell)$ and multiplies those weights by the selected value coordinates. The output should have shape $(1, 1, 4, 4)$, and the log-sum-exp tensor should have shape $(1, 4, 1)$. A test that checks only the output misses a required return value because later code or debugging tools may rely on the returned log-sum-exp.

Dense decode uses a different address path. It reads cache sequence lengths, recovers physical blocks from the block table for each valid prefix, reshapes those blocks into token rows, and then applies standard attention. If causal masking is enabled for a multi-query decode shape, the reference must apply the same causal convention as the kernel. In the recorded dense decode tests, correctness cases vary batch size, query length, cache length, key/value head count, variable-length cache rows, and causal mode. Those cases are not decorative; they protect the shape and masking invariants that a serving runtime depends on.

The pinned FlashMLA tests use source-specific combinations of absolute, relative, and sometimes cosine-difference tolerances. They remain attached to the tested dtype, output field, mode, and shape rather than becoming universal thresholds.

The FP8 cache path needs an additional round-trip test. A quantized cache row should dequantize to the expected BF16-like representation under the documented scale convention, and the RoPE component should remain in the documented higher-precision portion of the layout. A correctness report that measures only end-to-end attention output may miss a layout bug that is compensated by a particular random input.

Dense prefill adds a gradient oracle. The pinned FlashMLA test uses a variable-length PyTorch attention calculation as its reference, applies the same fixed upstream output gradient, and compares all three input gradients. It also checks repeated forward output and LSE equality and repeated d_k and d_v equality for its selected cases, while deliberately not asserting bitwise equality for d_q. That test behavior does not override the public wrapper's rejection of `deterministic=True`; it simply records what the source test establishes at the pinned revision.

The July 9 local H200 sweep covers the recorded fixed-shape FlashMLA correctness cases. The run uses FlashMLA commit `9241ae3ef9bac614dd25e45e507e089f888280e0`, PyTorch 2.11.0 + cu128, CUDA 12.8, and one NVIDIA H200. It does not claim broader shape coverage or serving-runtime selection. Table 22.13 records the closed surfaces.

Table 22.13.: Fixed-shape correctness results for dense decode, sparse decode, sparse prefill, and FP8 KV-cache layout on one H200.

Correctness surface	Completed local rows	Result boundary
Dense decode	Two rows: $b = 1, s_q = 1, s_k = 256, h_q = 16$ and $b = 2, s_q = 1, s_k = 512, h_q = 64$.	Output and log-sum-exp allclose predicates passed for the recorded block-table cache layouts.
Sparse decode	Two rows with $s_{kv} = 512$, top-$k = 64$, and one extra sparse context case.	The FlashMLA sparse-decode reference predicate returned `is_correct=true`.
Sparse prefill	Two rows with top-$k = 128$, including an attention-sink and top-k-length case.	The FlashMLA sparse-prefill predicate returned `is_correct=true`.
FP8 KV-cache round trip	Two layouts: `V32_FP8Sparse` and `MODEL1_FP8Sparse`.	RoPE coordinates were preserved exactly; non-RoPE dequantization max absolute error was 0.015625 in both rows.

The July 20 packet extends only the quantized sparse-cache row, rather than replaying the dense, sparse-decode, or sparse-prefill timing sweep. It adds four V32 and three MODEL1 fixtures across irregular block sizes, all-invalid indices, an oversized top-k, a second cache scope, a sink, and injected tile outliers. All seven passed the output and LSE gates adopted from the pinned FlashMLA sparse-decode tests, and every RoPE payload was bitwise preserved. Table 22.14 reports maxima across finite pairs; the reviewed record retains every case's absolute, relative, anomaly-mask, and ordered-bit ULP-style distributions.

Table 22.14.: Expanded one-H200 FlashMLA quantized-cache comparisons at commit `9241ae3ef9ba`. Maxima aggregate the recorded finite pairs without turning them into universal tolerances.

Comparison	Maximum absolute error	Scope
Non-RoPE cache round trip	0.0234375	Nine primary/extra cache scopes; RoPE payloads were checked bitwise, not with a numerical tolerance.
Kernel output versus dequantized-cache reference	0.000244141	Seven output gates across V32 and MODEL1 layouts.
Kernel LSE versus dequantized-cache reference	1.43×10^{-6}	Finite LSE pairs; all-invalid cases retain exact nonfinite-mask comparisons.
Kernel output versus original BF16-cache reference	0.00128174	Includes both cache quantization and kernel-versus-reference effects.
Kernel LSE versus original BF16-cache reference	0.000436306	Same combined-effect boundary as the preceding row.

22.14. Benchmarking and Integration

A FlashMLA timing row is comparable only when kernel mode, attention shape, cache format, hardware, and correctness gate match. The project publishes benchmark commands, hardware assumptions, CUDA requirements, and performance claims [27]. The generic measurement record is defined in the inference-cost chapter; the evaluation harness appendix explains how to implement its artifact trail. Table 22.16 below keeps only the fields that specialize that record for FlashMLA: kernel mode, attention shape, cache format, command, correctness gate, and derived metrics.

Definition 22.14.1 (Integration responsibility split) *An integration responsibility split divides work between the kernel library, the call site that prepares its tensors, and the serving runtime that owns request-level state. FlashMLA's responsibility is narrow: the kernel computes a specified attention output and log-sum-exp for supplied tensors, while the runtime remains responsible for batching, cache-page allocation, page-table updates, sparse-index construction, sampling, and user-visible error handling. End-to-end serving correctness requires all three layers [27, 50].*

The benchmark card identifies the operation that ran; the responsibility split identifies the owner of each result. The toy operation count from the decode section illustrates the denominator. For dense decode with $B = 1$, $S_q = 1$, $H_Q = 4$, $T = 4$, $d_{qk} = 6$, and $d_v = 4$, the score and value products account for about 320 multiply-add flops under the simple $2mnk$ convention. Sparse top-k with $k = 2$ accounts for about 160 such flops. A comparable timing row states whether it used $T = 4$ or $k = 2$, BF16 or FP8 cache rows, dense or sparse mode, and which hardware ran the kernel. The same rule scales to FlashMLA's production-sized benchmark shapes.

The pinned repository supplies useful release calibration before the local H200 rows. Table 22.15 keeps those headline maxima in their source-reported class. The README names the benchmark commands, mode, cache format, GPU, and CUDA version. Because the maximizing tensor shape is not printed for every value, the rows provide release orientation; the local H200 measurements use different shapes and timing protocols.

Released surface	REPORTED maximum	README coordinate	Supported use
Dense MLA decode	Up to 3000 GB/s when memory-bound; up to 660 TFLOP/s when compute-bound	H800 SXM5, CUDA 12.8, dense MQA with BF16 cache	The release reports maxima without one printed maximizing shape.
Sparse MLA decode	Up to 410 TFLOP/s when compute-bound	H800 SXM5, CUDA 12.8, FP8 KV cache with BF16 matrix multiplication	This is the sparse-crossover family, not the dense-decode bandwidth row.
Sparse MLA prefill	Up to 640 TFLOP/s in forward computation	H800 SXM5, CUDA 12.8, sparse MLA prefill benchmark	Prefill and decode have different query shapes and timing surfaces.

Table 22.15.: Performance maxima reported by the FlashMLA README at commit `9241ae3e`. They provide release orientation; the local H200 rows use different shapes and timing protocols.

The benchmark card also keeps correctness and performance connected. Table 22.16 lists the minimum fields needed before a run can be used as evidence.

Field	Recorded value or requirement
Source revision	FlashMLA commit `9241ae3ef9bac614dd25e45e507e089f888280e0`
Kernel mode	Dense decode, sparse decode, or sparse prefill
Shape	$B, S_q, S_{kv}, H_Q, H_{\mathrm{KV}}, d_{qk}, d_v, P$, `topk`
Cache format	BF16 dense cache or documented FP8-with-scale sparse layout
Command	Exact test or benchmark command and arguments
Correctness gate	Reference output fields and tolerance policy passed before timing is interpreted
Metrics	Latency, derived flops, derived bytes, bandwidth, throughput, and the formulas used
Artifact	Raw terminal log or profiler output path

Table 22.16.: Minimum benchmark card for turning a FlashMLA run into evidence.

Two representative local rows close the benchmark argument in its owning chapter. Both passed their output and LSE gates before timing. They share one H200 and the same five-warmup, 50-launch, five-repeat-mean policy, but they execute different attention phases and shapes. Table 22.17 therefore presents each as an orientation row.

Correctness-gated H200 path	OBSERVED latency	Local scope
Sparse decode: $(b, s_q, s_{kv}, K, h_q, d_{qk}) =$ $(4, 1, 512, 64, 64, 576)$, V32 FP8 cache	0.0993 ms mean; 0.1008 ms median	Hot-cache standalone kernel calls; request-level timing belongs to runtime measurements
Sparse prefill: $(s_q, s_{kv}, K, h_q, d_{qk}) =$ $(1, 128, 128, 64, 576)$, BF16	0.0451 ms mean; 0.0450 ms median	Standalone sparse-prefill calls in the baseline mode that omits sink and top-k length

Table 22.17.: Representative local FlashMLA measurements on one H200. The complete microbenchmark matrix and cross-system diagnostics remain in the NVIDIA Kernels and Runtime Measurements chapter.

The recorded H200 dense, sparse-decode, and sparse-prefill timings all use the benchmark card, but their protocols remain separate. The RTX 5090 and Jetson Thor runs instead resolve build, cache-format, and architecture-guard boundaries.

The complete H200 matrix and those diagnostic states remain in Chapter 35. The H200 timing measures execution, the SM120 guard tests eligibility, and the SM110 cache round trip tests format handling.

The final boundary is integration. FlashMLA provides kernels and Python bindings. A serving runtime still owns request admission, batching, cache-page allocation, block-table maintenance, sparse-index construction, model-forward wiring, logits processing, sampling, error handling, and observability. Kernel correctness establishes the attention result for one specified call. End-to-end serving correctness additionally requires compatible batching, synchronized page tables, intended sparse positions, and runtime failure handling.

Table 22.18 separates the kernel's responsibility from the surrounding runtime responsibilities.

Table 22.18.: Responsibility split for a runtime that uses FlashMLA.

Component	Responsibility
Serving runtime	Request scheduling, batching, cache allocation, page-table updates, sparse index construction, model-forward orchestration, sampling, and user-visible error handling.
FlashMLA call site	Convert runtime state into the documented tensor shapes, mode flags, scheduler metadata, and address metadata expected by the kernel.
FlashMLA kernel	Compute the specified attention output and log-sum-exp for the supplied tensors and mode requirements.
Correctness harness	Compare kernel return values with a reference implementation under documented tolerances.
Benchmark harness	Time a correctness-gated call and report normalized metrics with complete configuration.

The auditable flow converts runtime state into a FlashMLA call record; the returned values must pass a reference comparison before benchmark metrics become chapter evidence.

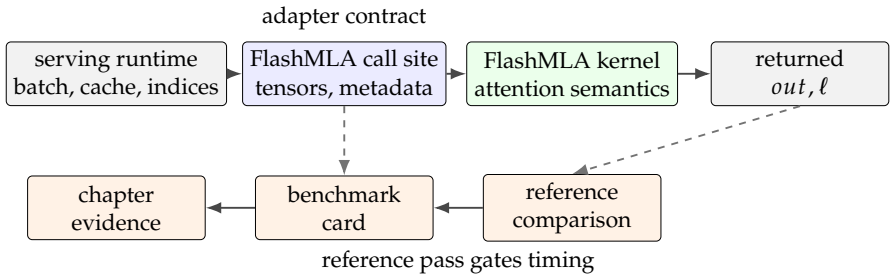

Figure 22.15.: A FlashMLA integration claim needs both sides of the responsibility split: runtime-owned request state must become a valid call record, and benchmark evidence must be gated by reference comparison of the returned values.

Figure 22.15 places reference comparison between the kernel return and the benchmark claim. SGLang makes the FlashMLA integration responsibility split concrete rather than leaving it implicit in the kernel API. The runtime registry maps the `flashmla` backend name to `FlashMLABackend`. `ForwardBatch` carries request rows, request-pool indices, sequence lengths, and output cache locations. `FlashMLABackend` turns that state into the FlashMLA `block_table`, cache-sequence lengths, scheduler metadata, and number-of-splits tensors [72]. The

call-site walkthrough in Table 22.19 follows that state from the request batch through the adapter and back into token-row order.

Table 22.19.: SGLang FlashMLA call-site walkthrough at the source-record snapshot. The table maps source behavior; timings come from executed runs.

SGLang step	Source record and fields	FlashMLA-facing consequence
Backend selection	the recorded SGLang source registers `flashmla`; the recorded SGLang source contains DeepSeek attention-backend selection logic.	The runtime can select the FlashMLA adapter, but a smoke run must still prove the selected model and hardware take that path.
Batch handoff	the recorded SGLang source describes `ScheduleBatch->ForwardBatch`; the batch carries `req_pool_indices`, `seq_lens`, `seq_lens_cpu`, and `out_cache_loc`.	Request rows and cache locations are explicit runtime metadata before the kernel call.
Page-table build	the recorded SGLang FlashMLA source uses `req_to_token`, `req_pool_indices`, and `seq_lens` to fill `block_kv_indices` with `create_flashmla_kv_indices_triton`.	The FlashMLA `block_table` is derived from SGLang request/cache state, not constructed by the kernel.
Scheduler metadata	The same adapter calls `get_mla_metadata` with sequence lengths, local query-head count, and `is_fp8_kvcache`.	FlashMLA receives metadata and `num_splits`; FP8 KV-cache interpretation is part of the call record.
Kernel call and output row	`forward_decode` calls `flash_mla_with_kvcache` with reshaped query, latent KV cache, `block_table`, `cache_seqlens`, metadata, and softmax scale.	The returned attention output is reshaped back to the runtime's token-row order before the model-forward path continues.
Fallback and graphs	Non-decode extend paths call the `FlashInferMLAAttnBackend` parent; CUDA graph state stores `cuda_graph_kv_indices`, metadata, and `num_splits`.	Prefill/fallback and graph-capture claims must be checked separately from dense decode.

Source inspection maps how SGLang prepares FlashMLA inputs. The retained one-H200 diagnostics add a bounded backend-selection result: an FA3 negative control and FlashMLA candidate establish dispatch attribution, while a controlled follow-up compares native and FP32-reference outputs on identical wrapper inputs and records fallback plus selected-kernel identity. The exact gates and observed values are consolidated in Chapter 35. That dossier keeps dispatch, same-input comparison, fallback, and selected-kernel identity as independent gates.

Two interpretive rules remain local because they belong to FlashMLA integration. First, an installed SGLang kernel binding is a versioned runtime surface; equivalence to the standalone FlashMLA revision requires an explicit version check. Second, token agreement and tensor-level equivalence are independent gates: a small logit change may preserve one greedy decision or flip another near an argmax boundary. Runtime dispatch, tensor equivalence, fallback, request completion, and quality therefore remain separate claims.

22.15. Summary

FlashMLA preserves attention while changing cache representation, addressing, and reduction. Dense decode follows every valid logical position; sparse decode follows a selected set. The 656-byte FP8 sparse layout retains BF16 RoPE state, and Hopper reconstructs BF16 matrix operands with FP32 accumulation. DSA owns the learned choice of that selected set, FlashMLA owns its index-consuming execution, and standard dense FlashAttention reduces materialization and data movement while retaining the dense legal-position set. FlashAttention-3 changes the Hopper execution schedule and precision options; DSA and FlashMLA retain selection ownership.

Crossover divides each 64-position block into two conversion assignments; seesaw divides a large output accumulator between warpgroups while preserving one online-softmax state. TMA arrival, matrix completion, CTA handoff, and split reduction each have their own completion condition. Dense decode, sparse FP8 decode, sparse MLA prefill, and dense MHA prefill expose four interfaces.

In the inspected SM100 head-128 sparse-prefill path, four-row TMA gathers stage complementary key and value partitions for two CTAs, and paired `tcgen05` operations form score and value products. Its softmax state stores the true maximum independently from an exponent anchor and opens the collective rescale path only when some row has a base-2 anchor gap greater than six. Deferred rescaling changes tensor-memory work while preserving normalized output and LSE. Absorbed decode reuses one latent-plus-RoPE row across heads; measured ridge data and clocks classify compute or bandwidth pressure.

The H200 runs establish fixed-shape correctness, selected timings, automatic fallback, and bounded SGLang route attribution. A same-input comparison matched request tokens while retaining elementwise output failures. Backend selection and output equivalence therefore require their own checks. The compiled artifact and launch trace connect emitted instructions to runtime selection. Chapter 23 applies the same method to projection and grouped-expert products.

22.16. Exercises

Use Contract 22.2.3 when an exercise places sparse-cache slots on the SM100 path.

Use Validation Rule 22.3.5 as the claim guard for FP8 cache layout and tolerance exercises.

1. Core: With page size $P = 2$ and block-table row $[7, 3]$, encode logical positions 0:3 as `indices_in_kvcache`, then select $\{1, 3\}$. From this trace, explain why dense decode needs a block table and cache lengths while sparse decode can consume physical slots directly.
2. Core: For two packed dense prefill sequences, state the shapes of q, k, v, o, ℓ. Then state the shapes of d_q, d_k, d_v and the two cumulative-length arrays. Explain why forward GQA support does not imply that the pinned backward wrapper accepts unequal query and KV head counts.

3. Challenge: Give a formal distinction between MLA cache compression and token-level sparse attention. Construct one example that changes the cache representation without changing the attended token set, and one example that changes the attended token set without changing the representation.
4. Core: Reproduce the 656-byte FP8 sparse KV-cache calculation from the documented FlashMLA layout. Which component remains BF16, and why does that matter for interpreting "FP8 KV cache"?
5. Core: For absorbed decode with $h_q = 128$, $s_q = 1$, $d_k = 576$, and $d_v = 512$, calculate $h_q s_q (d_k + d_v)/d_k$ and compare it with $2h_q s_q$. Why does neither classify every call as compute-bound? For a 64×512 FP32 accumulator, calculate the registers for one and two full copies and each 64×256 seesaw half; state their shared softmax invariant.
6. Applied: Trace one Hopper crossover block with 64 selected positions, 128 heads, and 512 non-RoPE coordinates per row (the region FlashMLA calls NoPE). Give each CTA's head and row ownership, conversion counts before and after sharing, DSM transfer, readiness barrier, and invariant attention domain.
7. Applied: Trace one regular SM100 head-128 sparse-prefill block. Give K-row and V-coordinate ownership, the meaning of `gather4`, and readiness gates for QK^T and SV. Assuming no peer triggers first, use anchor $a = 10$ and block maxima $13, 17$ to find the cohort vote, anchor moves, total rescale, and equivalence to eager normalization.
8. Challenge: Write a two-scope sparse-decode reference. Apply each scope's valid length, then concatenate the surviving scores. If both scopes are nonempty, merge their partial LSE values stably. Normalize jointly, apply a finite attention sink, and explain why the returned LSE excludes the sink.
9. Applied: Design a correctness test for sparse decode that checks out, ℓ, scheduler-length reuse, and the SM100 allocation precondition. Include one slice of a contiguous allocation and one disjoint-page fixture that the adapter must reject before launch.
10. Research: A release reports two kernel throughput numbers from different GPUs or different sequence shapes. List the benchmark configuration fields required before comparing them, and explain one invalid comparison.
11. Challenge: **Source analysis.** What may `FlashMLASchedMeta` cache? Name a TMA-staged object, WGMMA product, stale-metadata split-combine failure, and PDL readiness rule. For a built SM90 extension, state what `cuobjdump` and `nvdisasm` can establish about PTX, cubin, SASS, resources, and an absent instruction pattern.
12. Applied: Sketch or tabulate the responsibility split for a serving runtime using FlashMLA. Mark which component owns request batching, KV-page allocation, sparse index construction, kernel invocation, output validation, sampling, and error handling.